INTEGRATED PROJECT REPORT

On

Evaluating Redundancy and Compression Limits in Convolutional Neural Networks for Edge Deployment

Submitted in Partial Fulfilment of the Requirements for the Degree of

Bachelor of Technology

In

Artificial Intelligence & Data Science

By

Submitted By:

Sagar Sharma (03015611924)

Kush Lodhi (01215611924)

Vishesh Idnani (02415611924)

Under the Supervision of

Ms. Meenu Sharma

Assistant Professor

Department of Artificial Intelligence & Data Science

Dr. AKHILESH DAS GUPTA INSTITUTE OF PROFESSIONAL STUDIES

(A Unit of BBD Group)

Approved by AICTE and Affiliated with GGSIP University

FC-26, Shastri Park, New Delhi-110 053

[Academic Year 2025-26]

DECLARATION

We, Sagar Sharma, Kush Lodhi, and Vishesh Idnani, hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

The work presented in this report titled "Evaluating Redundancy and Compression Limits in Convolutional Neural Networks for Edge Deployment" has been carried out by us under the supervision of Ms. Meenu in the Department of Artificial Intelligence & Data Science, Dr. Akhilesh Das Gupta Institute of Professional Studies, New Delhi, during the academic year 2025-26.

We also declare that no part of this work has been reproduced from any other published source without proper citation.

Signature: _____

Name: _____

Roll No.: _____

Date: _____

Signature: _____

Name: _____

Roll No.: _____

Date: _____

Signature: _____

Name: _____

Roll No.: _____

Date: _____

CERTIFICATE

This is to certify that the Project Report entitled "Evaluating Redundancy and Compression Limits in Convolutional Neural Networks for Edge Deployment " which is submitted by Sagar Sharma, Kush Lodhi, and Vishesh Idnani in partial fulfilment of the requirement for the award of degree B.Tech. in the Department of Artificial Intelligence & Data Science of Dr. Akhilesh Das Gupta Institute of Professional Studies (ADGIPS), formerly known as Dr. Akhilesh Das Gupta Institute of Technology & Management (ADGITM), New Delhi, is a record of the candidate's own work carried out by them under our supervision. The matter embodied in this thesis is original and has not been submitted for the award of any other degree.

Date: _____

Supervisor Signature
Ms. Meenu Sharma

HOD Signature
Dr. Archana Kumar

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our project supervisor, Ms. Meenu , for the invaluable guidance, consistent feedback, and encouragement provided throughout the course of this research. Their insights into model compression, deep learning systems, and experimental methodology significantly shaped the direction and quality of this work.

We are grateful to the Department of Artificial Intelligence & Data Science at Dr Akhilesh Das Gupta Institute of Professional Studies for providing us with the academic environment and resources necessary to carry out this research.

We also thank Dr. Archana Kumar, Head of the Department of AI & Data Science, for facilitating the practicum research programme and maintaining high academic standards.

This project made use of Google Colab's freely available T4 GPU infrastructure, without which training experiments would have been substantially slower. We are also grateful to Weights & Biases (wandb.ai) for providing a free academic tier that enabled structured experiment tracking across all phases of this project.

Finally, we thank our families and peers for their patience and moral support during the duration of this work.

Signature: _____

Name: _____

Roll No.: _____

Date: _____

Signature: _____

Name: _____

Roll No.: _____

Date: _____

Signature: _____

Name: _____

Roll No.: _____

Date: _____

ABSTRACT

Deep neural networks have demonstrated remarkable accuracy on image classification benchmarks, yet their deployment on edge devices — microcontrollers, mobile phones, and single-board computers — remains constrained by model size, inference latency, and memory footprint. This paper presents a systematic empirical study of three compression techniques — unstructured magnitude pruning, structured channel pruning, and post-training static quantization — applied to ResNet-18 trained on CIFAR-10, with knowledge distillation evaluated as a complementary training strategy. The baseline ResNet-18 (CIFAR-adapted) achieves 95.34% top-1 accuracy at 44.77 MB with 28.2 ms CPU inference latency. Unstructured pruning reveals that 70% of weights are redundant and can be zeroed with only 0.4% accuracy loss, but produces no measurable reduction in model size or latency due to the absence of hardware-level sparse compute support. Structured channel pruning at 50% removal, combined with an extended fine-tuning protocol (CutMix augmentation, cosine annealing, 40 epochs), recovers to 94.25% accuracy at 1.62× speedup and 10.73 MB. Applying post-training static INT8 quantization to this compressed model yields a hero result: 94.32% accuracy, 2.07× faster inference, and 2.76 MB on-disk footprint — a 15.5× size reduction from baseline with under 1% accuracy cost. Knowledge distillation using a ResNet-50 teacher does not improve upon the fine-tuned structured model, demonstrating that capacity constraints in heavily pruned architectures limit the effectiveness of soft-target supervision. All experiments are reproducible via publicly available code and model checkpoints. These findings offer practical guidance for practitioners seeking to deploy convolutional neural networks on CPU-constrained edge hardware.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
BN	Batch Normalization
CIFAR-10	Canadian Institute for Advanced Research – 10 Class Dataset
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DNN	Deep Neural Network
FLOPs	Floating Point Operations per Second
FP32	32-bit Floating Point Representation
GPU	Graphics Processing Unit
INT8	8-bit Integer Representation
IoT	Internet of Things
KD	Knowledge Distillation
KL	Kullback-Leibler (divergence)
LTH	Lottery Ticket Hypothesis
MACs	Multiply-Accumulate Operations
ML	Machine Learning
PTQ	Post-Training Quantization
QAT	Quantization-Aware Training
RAM	Random Access Memory
ReLU	Rectified Linear Unit
SGD	Stochastic Gradient Descent
WandB	Weights & Biases (experiment tracking platform)

CHAPTER 1

Introduction

1.1 Introduction of the Problem

Modern deep learning has produced convolutional neural networks (CNNs) capable of exceeding human-level accuracy on standardized image classification benchmarks. Models such as ResNet, VGG, and EfficientNet achieve state-of-the-art results by scaling depth, width, and resolution — but this performance comes at a cost. A standard ResNet-18 contains approximately 11 million parameters and requires over 550 million multiply-accumulate operations (MACs) for a single forward pass on a 32×32 input image. At full 32-bit floating-point precision, such a model occupies nearly 45 megabytes of memory. These characteristics are well within the capabilities of server-grade GPUs, but they present a serious obstacle for deployment on edge devices.

Edge devices encompass a broad category of hardware: smartphones, Raspberry Pi-class single-board computers, microcontroller units (MCUs), IoT sensors, and embedded systems found in automotive, medical, and industrial applications. These devices are characterized by constrained memory (often 256 KB to 1 GB RAM), limited compute throughput (no dedicated GPU, often a single-core or low-power ARM/x86 CPU), and strict energy budgets. Deploying a 45 MB model onto such hardware is either infeasible due to memory limits, or impractically slow due to the absence of hardware-accelerated matrix operations.

The gap between the computational demands of modern deep learning models and the hardware constraints of edge devices motivates a family of techniques collectively referred to as model compression. These techniques aim to reduce the size, compute cost, and memory footprint of neural networks without proportional degradation in predictive accuracy. The three dominant approaches are pruning (removing redundant parameters), quantization (reducing numerical precision), and knowledge distillation (training a smaller model to mimic a larger one). Each technique targets a different axis of inefficiency and comes with distinct trade-offs in implementation complexity, accuracy preservation, and hardware compatibility.

This project addresses a specific and practically important question: how far can ResNet-18 be compressed before accuracy meaningfully degrades, and do theoretical efficiency gains translate into measurable CPU inference improvements? ResNet-18 is not a large model by modern

standards, yet it is substantially overparameterized for the CIFAR-10 classification task — 11 million parameters for a 10-class problem with 32×32 images. This overparameterization is intentional: it creates the redundancy that compression techniques exploit, and it produces accuracy-sparsity curves that are legible and interpretable.

The theoretical motivation for this investigation draws from two foundational works. The Lottery Ticket Hypothesis (Frankle & Carlin, 2019) posits that dense, randomly initialized neural networks contain sparse subnetworks — termed 'winning tickets' — that, when trained in isolation with their original initialization, can match or exceed the accuracy of the full network. This framing implies significant structural redundancy in standard training pipelines and motivates the aggressive pruning experiments conducted in this study. Deep Compression (Han et al., 2016) demonstrated that a three-stage pipeline — pruning, quantization, and Huffman encoding — can compress models by 35× to 49× with negligible accuracy loss, establishing a practical benchmark for what is achievable through systematic compression.

Unlike prior work that evaluates these techniques in isolation or on large-scale benchmarks (ImageNet, COCO), this study implements a clean, reproducible pipeline on CIFAR-10 using a single architecture family, documenting both positive results and negative findings honestly. The flat size and latency curves produced by unstructured pruning, for instance, are not presented as failures but as empirical confirmation of a known theoretical limitation — and as motivation for the structured pruning phase that follows.

Table 1.1: Summary of edge deployment hardware constraints

Device Class	RAM	Compute	Constraint
MCU / Sensor	256 KB – 1 MB	< 100 MHz ARM	No OS, no FP
Raspberry Pi Zero	512 MB	1 GHz ARM single-core	No GPU
Raspberry Pi 4	1–8 GB	1.8 GHz ARM quad-core	CPU inference only
Android Phone	2–8 GB	Mobile ARM SoC	Battery & thermal
Edge AI Accelerator	2–4 GB	NPU (limited ops)	Fixed op support

1.2 Summarize Previous Research

The problem of deploying large neural networks on resource-constrained hardware has attracted significant research attention since deep learning began dominating computer vision benchmarks in 2012. The field of model compression has matured considerably over the past decade, producing a rich body of work spanning pruning, quantization, knowledge distillation, neural architecture search, and hardware-software co-design. This section summarizes the most relevant prior work.

The earliest systematic work on neural network pruning dates to LeCun et al. (1990), who proposed Optimal Brain Damage — using second-order derivative information (the Hessian) to identify and remove parameters with low impact on the loss. Hassibi and Stork (1993) extended this to Optimal Brain Surgeon, which achieved higher accuracy retention through more precise saliency estimation. However, the computational cost of Hessian computation made these methods impractical for large networks. The modern era of pruning was reinvigorated by Han et al. (2015), who showed that simple magnitude-based pruning — removing weights below a threshold — could reduce AlexNet and VGG-16 parameter counts by 9× to 13× with no loss in top-5 accuracy on ImageNet.

Deep Compression (Han et al., 2016) extended this work into a full three-stage pipeline: magnitude pruning followed by weight sharing (trained quantization via k-means clustering) and Huffman coding. Applied to AlexNet and VGGNet, Deep Compression achieved 35× and 49× compression respectively with negligible accuracy loss. This work established the principle that pruning and quantization are complementary — each addressing a different axis of inefficiency — and that their combination produces compression ratios far exceeding what either can achieve independently.

The Lottery Ticket Hypothesis (Frankle & Carlin, 2019) reframed pruning as a search problem rather than a compression problem. The authors demonstrated that standard dense networks trained with SGD contain sparse subnetworks — winning tickets — that, when isolated and retrained from their original initialization, match full network accuracy. This finding implies that the full network is, in a meaningful sense, a collection of subnetworks of which only a small fraction are functionally necessary. Subsequent work by Frankle et al. (2020) extended this to larger architectures, finding that the lottery ticket phenomenon persists in ResNets, though

identifying winning tickets at scale requires rewinding to an early checkpoint rather than random initialization.

Structured pruning, which removes entire channels, filters, or layers rather than individual weights, emerged as a practical response to the hardware limitation that unstructured sparsity does not reduce real-world FLOPs unless sparse compute is hardware-supported. Li et al. (2017) introduced filter pruning using L1-norm as a saliency criterion, demonstrating that removing complete output feature map channels produces smaller, faster models with only moderate accuracy loss. He et al. (2017) proposed soft filter pruning, allowing temporarily zeroed filters to recover through continued training. More recent work by Fang et al. (2023) introduced DepGraph, which models inter-layer dependencies as a graph and enables coupled pruning of groups of parameters that must be removed together — the same principle leveraged by the torch-pruning library used in this project.

Post-training quantization (PTQ) converts trained FP32 models to lower-precision representations — INT8, INT4, or even binary — without requiring retraining. Banner et al. (2019) demonstrated that careful calibration of activation clipping ranges enables INT8 PTQ with minimal accuracy loss. Nagel et al. (2020) introduced ADAROUND, a data-free method for optimizing rounding decisions in quantized weights. PyTorch's native quantization pipeline, used in this project, implements static PTQ using FBGEMM (x86) and QNNPACK (ARM) backends, with calibration-based activation scale estimation.

Knowledge distillation was introduced by Hinton et al. (2015) as a method for compressing ensemble models into a single smaller network. The key insight is that the soft probability distribution output by a teacher model — rather than the hard one-hot label — carries richer supervisory signal: it encodes the teacher's notion of inter-class similarity. A temperature parameter T controls the softness of these distributions, with higher T producing smoother probability vectors that reveal more structural information. Romero et al. (2015) extended this to intermediate layer matching (FitNets), allowing the student to mimic not just the teacher's output but internal representations. Park et al. (2019) introduced Relational Knowledge Distillation, transferring structural relationships between data points rather than point-wise soft targets.

Architecture-level efficiency was advanced by Howard et al. (2017) with MobileNetV1, which replaced standard convolutions with depthwise-separable convolutions, reducing parameter

counts by 8–9× with small accuracy penalties. Sandler et al. (2018) introduced MobileNetV2 with inverted residuals, and Howard et al. (2019) further refined this with MobileNetV3 using neural architecture search. Tan and Le (2019) proposed EfficientNet, demonstrating that compound scaling of width, depth, and resolution — guided by an NAS-discovered baseline architecture — achieves state-of-the-art accuracy at a fraction of the compute of prior models. While these architectures represent an alternative path to edge deployment, they make different trade-offs: they are harder to compress further (MobileNets are already near-optimal in parameter density), produce less interpretable compression curves, and require more complex training recipes.

On-device inference engines such as TensorFlow Lite, ONNX Runtime, and OpenVINO have standardized the deployment pathway for compressed models on edge hardware. These runtimes support INT8 and FP16 inference, operator fusion, and platform-specific optimization. TFLite's representative dataset API implements the same calibration principle as PyTorch's static PTQ. CoreML and the Android Neural Networks API provide vendor-specific paths for iOS and Android deployment respectively.

1.3 Researching the Problem

This research situates itself within the empirical benchmarking tradition of the compression literature, rather than the technique-novelty tradition. A significant body of prior work evaluates a single compression technique in depth on a single architecture. This project evaluates three techniques — unstructured pruning, structured pruning, and PTQ — in sequence on a single architecture, enabling direct comparison of their accuracy-efficiency trade-offs in a controlled setting.

The research approach drew from multiple threads of investigation. First, the theoretical foundations of each compression technique were studied to ensure correct implementation: Han et al. (2015) for magnitude pruning, Frankle & Carlin (2019) for the Lottery Ticket framing, Li et al. (2017) for structured pruning, and PyTorch official documentation for the quantization and distillation APIs. Second, practical implementation details were sourced from the PyTorch ecosystem: torch.nn.utils.prune for unstructured pruning, torch-pruning (DepGraph library) for structured pruning, and torch.ao.quantization FX graph mode for static PTQ.

A key research decision was the choice to use FX graph mode PTQ rather than eager mode. During implementation, static PTQ in eager mode failed with an aten::add.out dispatch error on

the QuantizedCPU backend, stemming from the residual addition operations in ResNet skip connections. Eager mode quantization requires explicit FloatFunctional wrappers for these additions — a modification that cannot be made to torchvision's standard ResNet implementation without rewriting the model class. FX graph mode resolves this by tracing the full compute graph and handling all operations, including residual additions, as first-class nodes in the quantization pipeline. This finding, while not the primary result of the study, is a practical contribution: it is not widely documented in tutorials, and practitioners replicating PTQ on ResNets are likely to encounter the same failure.

The literature on knowledge distillation for compressed models informed the design of the distillation phase. The choice to initialize the student from the fine-tuned structured-50% checkpoint — rather than random initialization — reflects the practical deployment scenario: the student already has a useful set of weights, and distillation is being applied to see if a better teacher signal can close the remaining accuracy gap. This initialization strategy also serves as a controlled experiment: if distillation fails to improve on the fine-tuned result, it implies that the accuracy ceiling for this compressed architecture is determined by its capacity (number of remaining channels), not its training signal quality.

All experiments were tracked using Weights & Biases, with consistent run naming conventions and metric schemas to enable clean comparison across phases. Inference latency was measured as the mean of 100 forward passes on a single CPU core, with 20 warmup runs discarded. Peak RAM was recorded using tracemalloc. Model size was measured as the on-disk checkpoint file size after each compression stage. These protocols are documented in Chapter 3 and reported consistently throughout Chapter 4.

CHAPTER 2

Literature Survey

2.1 Deep Learning for Image Classification

The modern era of deep learning for image classification began with AlexNet (Krizhevsky et al., 2012), which won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) by a margin of 10.9% over the second-place submission. AlexNet demonstrated that deep convolutional networks trained on large datasets with GPU acceleration could dramatically outperform feature-engineering pipelines. Subsequent years saw rapid architectural evolution: VGGNet (Simonyan & Zisserman, 2014) showed that depth alone — using 3×3 convolutions stacked uniformly — was a principal driver of accuracy; GoogLeNet (Szegedy et al., 2014) introduced the Inception module, achieving high accuracy with far fewer parameters than VGG through multi-scale feature extraction; and ResNet (He et al., 2015) introduced residual connections, enabling the training of networks 50 to 152 layers deep and establishing a new ILSVRC record at 3.57% top-5 error.

ResNet's residual connections address the degradation problem — the empirical observation that very deep networks trained with SGD exhibit higher training error than shallower counterparts, likely due to optimization difficulties rather than overfitting. By adding the input directly to the output of a block (a skip connection), residual networks provide gradient highways that stabilize training and enable far greater depth. This architectural insight is central to this project: the skip connections in ResNet-18 constrain how structured pruning can be applied, since channels pruned in a residual branch must be pruned consistently across the corresponding shortcut connection.

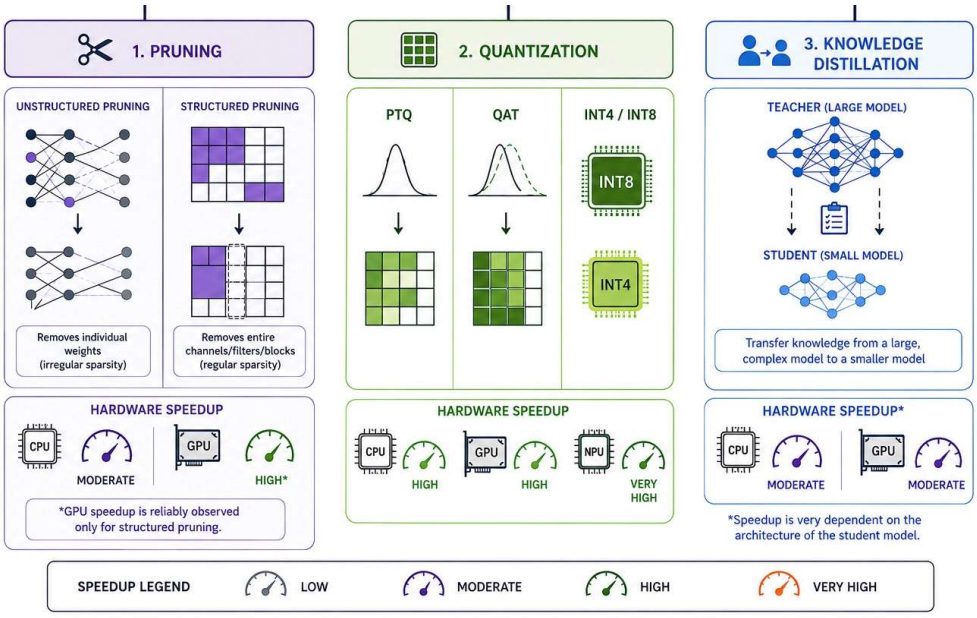

Figure 2.1: Taxonomy of neural network compression techniques

2.2 Model Compression — Motivation and Taxonomy

Model compression is motivated by the mismatch between the computational profile of state-of-the-art neural networks and the hardware budgets of target deployment platforms. This mismatch manifests on three axes: model size (parameter count and on-disk footprint), inference compute (FLOPs or MACs required per forward pass), and memory bandwidth (the rate at which weights must be loaded from DRAM into the compute units during inference). A compression technique that addresses one axis does not necessarily address the others — a fact that this project demonstrates empirically.

Unstructured pruning reduces the number of non-zero parameters and, in principle, model size if sparse storage formats are used. However, modern CPU and GPU hardware is optimized for dense matrix operations. Multiplying a sparse weight tensor by a dense activation tensor does not benefit from the zeros unless the hardware implements native sparse matrix-vector multiply (SpMV), which most consumer CPUs do not. As a result, unstructured pruning at 70% sparsity produces no measurable latency reduction on a standard x86 CPU — a finding confirmed in this study.

Structured pruning, by contrast, removes entire channels or filters, physically shrinking the tensor dimensions. A convolution layer with 64 output channels pruned to 32 output channels

produces activations of half the original spatial volume. Every subsequent operation — the next convolution, batch normalization, residual addition — operates on smaller tensors. This reduction in tensor volume directly reduces the number of MAC operations and the memory traffic, producing real FLOPs and real latency improvements. The cost is that structured pruning is more disruptive to the network than unstructured pruning: removing entire channels eliminates their contribution to all subsequent layers, making recovery through fine-tuning harder, especially at high pruning ratios.

Quantization trades numerical precision for reduced memory footprint and, when hardware INT8 compute units are available, for inference speedup. FP32 arithmetic uses 32 bits per weight; INT8 uses 8 bits — a 4× reduction in storage. On x86 CPUs, AVX-512 VNNI instructions enable vectorized INT8 multiply-accumulate operations, potentially offering 2–4× throughput improvement over FP32 on appropriately sized tensors. However, for very small models (below ~5M parameters), compute is rarely the bottleneck — data movement overhead, Python/PyTorch framework overhead, and memory access patterns dominate. This diminishing return of INT8 at small model sizes is another empirical finding of this project.

2.3 Pruning

The pruning literature spans several decades and multiple paradigms. Magnitude-based unstructured pruning — removing individual weights below a threshold — is the simplest and most widely studied approach. Han et al. (2015) showed that applying a global magnitude threshold to all layers, followed by fine-tuning, could reduce AlexNet parameters from 61M to 6.7M (9.1× compression) with no drop in top-5 accuracy on ImageNet. The torch.nn.utils.prune API used in this project implements this approach, applying a binary mask to weight tensors that zeroes out the pruned entries while preserving the full tensor structure.

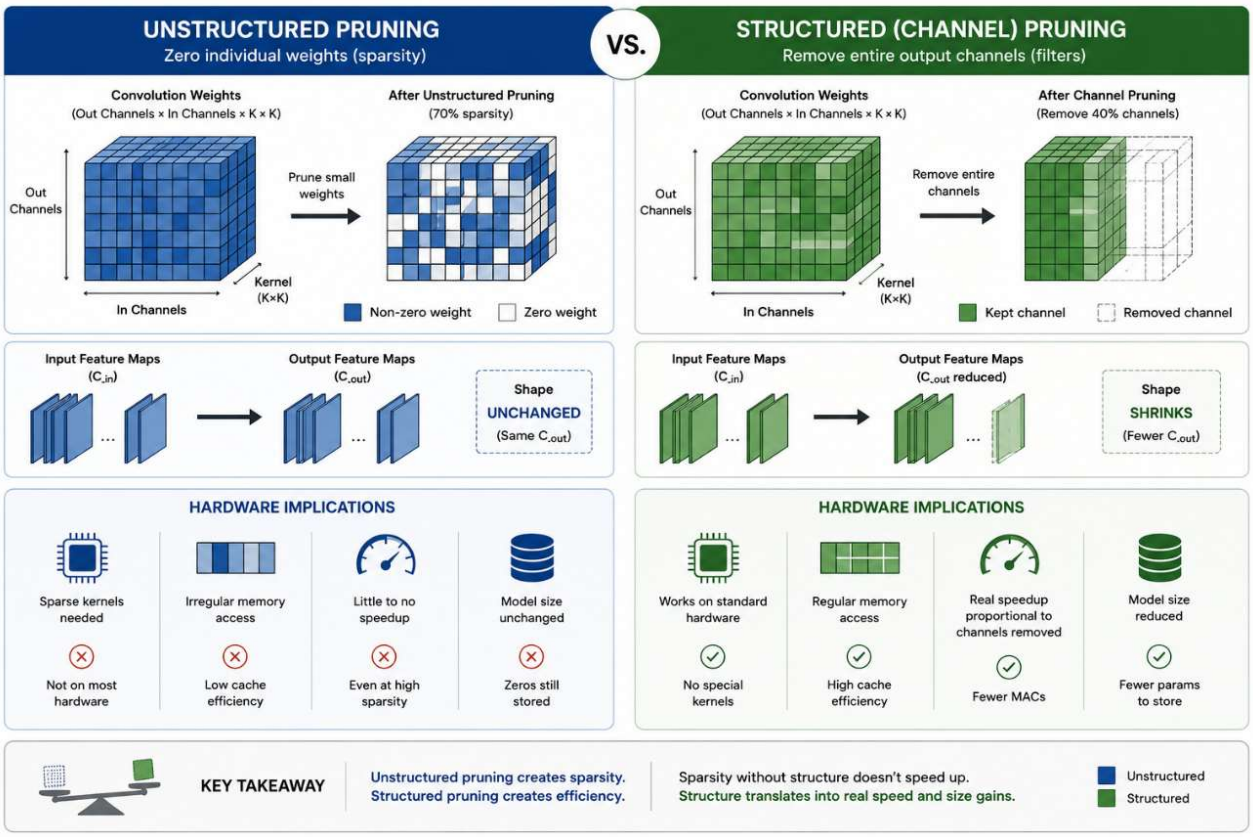

Figure 2.2: Unstructured vs. structured pruning — effect on tensor shape and hardware utilization

Structured pruning removes entire architectural units — channels, filters, attention heads, or layers. Li et al. (2017) used L1-norm of filter weights as a saliency criterion for filter-level pruning of VGG and ResNets, demonstrating that filters with small L1-norm contribute less to final accuracy and can be removed with limited damage. The key challenge in ResNets is dependency: the channels pruned in a convolutional layer must be pruned in the corresponding batch normalization layer and in the following layer's input channels, including across residual skip connections. The torch-pruning library (Fang et al., 2023) handles this through a dependency graph that identifies coupled parameter groups and ensures consistent pruning across branches.

Iterative magnitude pruning (IMP), used by Frankle & Carlin (2019) in the Lottery Ticket work, prunes a small fraction of weights at each iteration and retrains, repeating until the target sparsity is reached. IMP has been shown to consistently outperform one-shot pruning at the same final sparsity level, at the cost of significantly greater training budget. This project uses one-shot pruning for all experiments due to time constraints, with fine-tuning applied after each single pruning event.

Sensitivity analysis — evaluating the accuracy cost of pruning each layer independently — has been used to motivate non-uniform pruning schedules, where more tolerant layers are pruned more aggressively. Studies consistently find that earlier convolutional layers (closer to the input) and layers adjacent to skip connections are more sensitive to pruning than middle layers. This project applies a global uniform pruning ratio across all layers, which is the standard starting point for benchmarking studies.

2.4 Quantization

Quantization maps continuous FP32 weights and activations to discrete low-bit representations. The two principal variants are post-training quantization (PTQ), which converts a trained model without further training, and quantization-aware training (QAT), which simulates quantization during the forward pass during training, allowing the model to adapt its weights to the discretization error. PTQ is simpler to implement and requires no access to the training set beyond a small calibration dataset; QAT typically recovers 0.5–1.5% accuracy that PTQ cannot recover, but requires full training infrastructure.

Static PTQ — the variant used in this project — quantizes both weights (offline, at conversion time) and activations (online, using scales estimated from the calibration data). Dynamic quantization quantizes only weights offline, dequantizing activations at runtime; it is simpler to apply but does not reduce activation memory or achieve full INT8 speedup. Post-training quantization at INT8 typically degrades top-1 accuracy by less than 0.5% on well-trained classification models, a finding that is reproduced in this study across all structured-pruned variants.

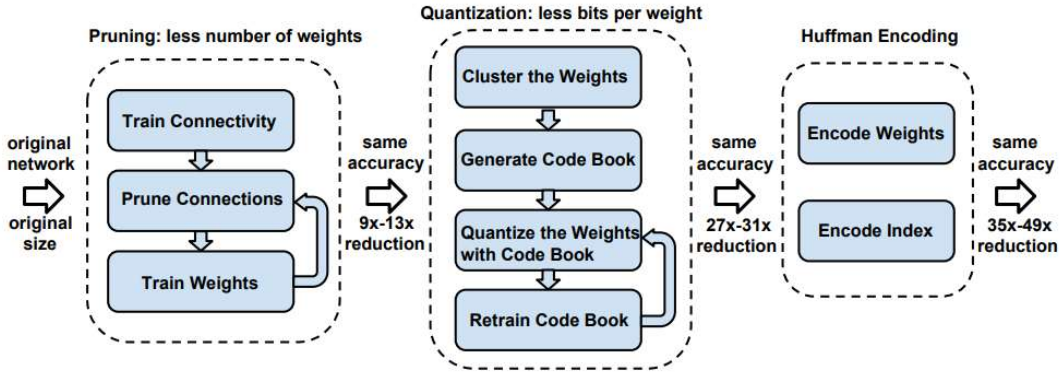

Figure 2.3: Deep Compression three-stage pipeline (Han et al., 2016)

The PyTorch quantization ecosystem provides two paths: eager mode, which requires manual insertion of QuantStub/DeQuantStub wrappers and FloatFunctional replacements for operations like addition; and FX graph mode, which traces the model's computation graph symbolically and applies quantization annotations to all nodes automatically. For architectures like ResNet where residual additions occur between module outputs, FX graph mode is substantially easier to apply correctly, as it handles the residual add nodes without requiring model source modification.

2.5 Knowledge Distillation

Knowledge distillation (Hinton et al., 2015) trains a student model using both the hard ground-truth labels and the soft probability distributions produced by a larger teacher model. The key insight is that the teacher's softmax output, particularly when softened with a temperature parameter $T > 1$, encodes rich information about inter-class relationships: for example, that a cat image is more similar to a dog than to an automobile, even if neither dog nor automobile has high probability. This inter-class structure, invisible in hard one-hot labels, provides a richer learning signal for the student.

The distillation loss is typically a weighted combination of the cross-entropy loss between student logits and ground-truth labels (hard loss) and the KL divergence between softened student and teacher probability distributions (soft loss). The weighting parameter alpha controls the relative contribution of each term. Common values are alpha = 0.7 for soft loss and (1 - alpha) = 0.3 for hard loss, which was the configuration used in this project.

Several extensions to the basic distillation framework have been proposed. FitNets (Romero et al., 2015) introduce intermediate layer matching, where the student is trained to reproduce internal feature map representations of the teacher, not just the final output. This is particularly useful when the student architecture is very shallow and the final output does not carry sufficient structural information. Attention Transfer (Zagoruyko & Komodakis, 2017) transfers activation statistics (attention maps) from teacher to student layers. Relational Knowledge Distillation (Park et al., 2019) transfers structural relationships between data points, capturing higher-order interactions that single-sample distillation misses.

For this project, the specific outcome of the distillation experiment is particularly informative: distillation from a ResNet-50 teacher did not improve on the extended fine-tuning

result (94.11% vs. 94.25%), suggesting that the compressed student's capacity — approximately 2.8M parameters after 50% structured pruning — is the binding constraint, not the quality of the training signal. This is consistent with the capacity-gap literature, which notes that very wide teacher-student gaps can harm student accuracy because the teacher's probability distributions are too confident to provide useful inter-class information.

2.6 Edge Deployment Constraints

The practical challenges of edge deployment extend beyond model size and latency. Quantized models require hardware INT8 support to achieve actual speedup: on x86 CPUs, INT8 compute benefits from AVX-512 VNNI or similar instructions; on ARM CPUs, SDOT instructions provide equivalent functionality. The PyTorch FBGEMM backend targets x86 CPUs specifically; QNNPACK targets ARM. On very small models, the overhead of the Python and PyTorch runtime often dominates the compute time, meaning that INT8 provides size reduction but not latency improvement.

Memory fragmentation, model loading time, and cold-start latency are additional deployment concerns not addressed by standard benchmarking protocols. The 100-run average used in this study measures steady-state inference latency on a pre-loaded model, which is appropriate for continuous-inference applications but does not capture the cost of model initialization in intermittent-inference scenarios (e.g., a camera that runs inference once per second).

TensorFlow Lite, ONNX Runtime, and OpenVINO provide more mature edge deployment runtimes than PyTorch's inference path, offering operator fusion, layout optimization, and hardware-specific kernel libraries. PyTorch's torch.ao.quantization pipeline targets server-side INT8 inference rather than edge deployment specifically; for Raspberry Pi deployment, a TFLite or ONNX export path would be preferred. This distinction is worth noting: the latency numbers reported in this study reflect PyTorch CPU inference in a Google Colab environment, not real device inference, and should be treated as relative comparisons rather than absolute deployment benchmarks.

CHAPTER 3

Methodology and Technology

3.1 Dataset and Preprocessing

All experiments in this study use the CIFAR-10 dataset (Krizhevsky & Hinton, 2009). CIFAR-10 consists of 60,000 RGB images of size 32×32 pixels, divided into 10 mutually exclusive classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The dataset is split into 50,000 training images and 10,000 test images, with each class represented equally at 6,000 images total (5,000 training, 1,000 test). There is no official validation split; following standard practice, the test set is used for evaluation.

Table 3.1: CIFAR-10 dataset statistics

Property	Training Set	Test Set	Total
Images	50,000	10,000	60,000
Number of Classes	10	10	10
Images per class	5,000	1,000	6,000
Image size	32×32 px	32×32 px	32×32 px
Colour Channels	RGB (3)	RGB (3)	RGB (3)

Training data augmentation follows standard CIFAR-10 practice: random horizontal flipping (p=0.5), random 4-pixel padding followed by random 32×32 crop (equivalent to a maximum 4-pixel translation in any direction), and normalization with channel-wise mean (0.4914, 0.4822, 0.4465) and standard deviation (0.2023, 0.1994, 0.2010). The extended fine-tuning protocol for the structured-50% model additionally applies CutMix augmentation (Yun et al., 2019) with a probability of 0.5 and alpha parameter 1.0. CutMix replaces a random rectangular region of one training image with the corresponding region from another image, mixing the labels proportionally to the area of each image's contribution. This form of data augmentation has been shown to improve fine-tuning recovery in compressed models by reducing overfitting to the compressed network's limited feature diversity. Test data undergoes only normalization — no augmentation.

3.2 Model Architecture

The baseline model is a CIFAR-10-adapted ResNet-18. Standard torchvision ResNet-18 was designed for ImageNet input resolution (224×224), using a 7×7 convolution with stride 2 and a 3×3 max-pooling layer with stride 2 as the stem — a total of 4× spatial downsampling before the first residual block. Applied directly to 32×32 CIFAR-10 images, this would reduce spatial dimensions to 4×4 before the first residual block, destroying spatial information.

To adapt ResNet-18 for CIFAR-10, two modifications were made to the stem: (1) the 7×7 stride-2 convolution is replaced with a 3×3 stride-1 convolution, preserving the spatial resolution at the first layer; and (2) the max-pooling layer is removed entirely. These two changes maintain 32×32 feature maps through the stem and into the first residual stage. The four residual stages of ResNet-18 then perform sequential 2× downsampling via stride-2 convolutions at their first block, ultimately producing 4×4 feature maps at the final stage. This adaptation is standard in the CIFAR-10 ResNet literature and produces competitive accuracy without architectural modification to the residual blocks themselves.

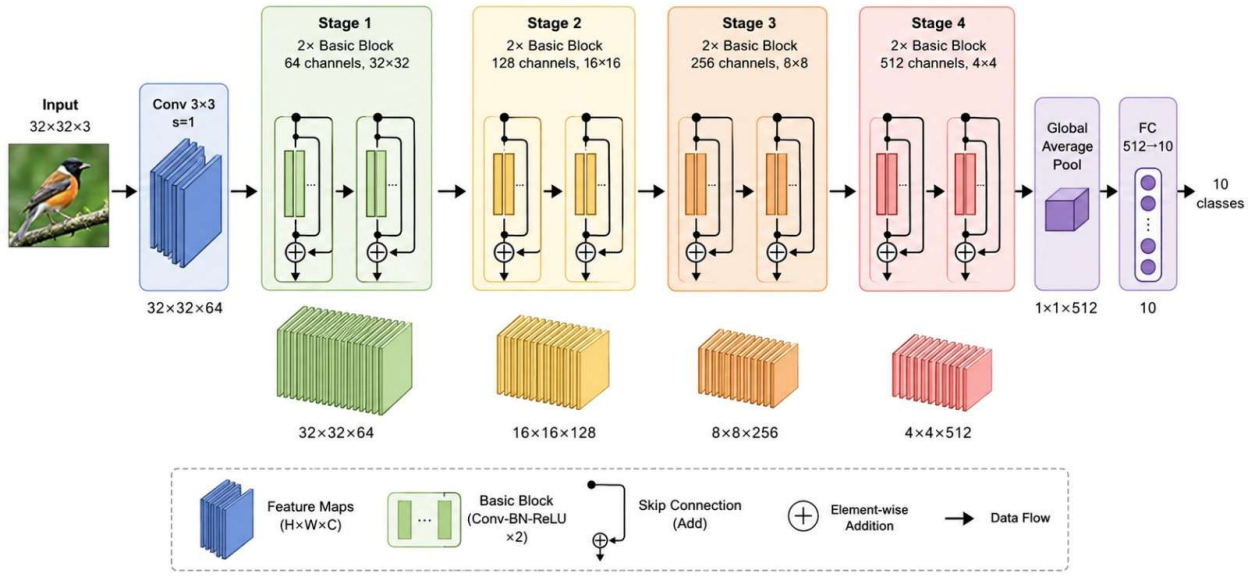

Figure 3.1: CIFAR-adapted ResNet-18 architecture — stem modification for 32×32 input

Table 3.2: ResNet-18 CIFAR-adapted architecture summary

Stage	Layer Type	Output Size	Channels	Blocks
Stem	Conv 3×3, BN, ReLU	32×32	64	—
Stage 1	BasicBlock × 2	32×32	64	2
Stage 2	BasicBlock × 2	16×16	128	2
Stage 3	BasicBlock × 2	8×8	256	2
Stage 4	BasicBlock × 2	4×4	512	2
Head	Avg Pool + FC	1×1	10 (logits)	—

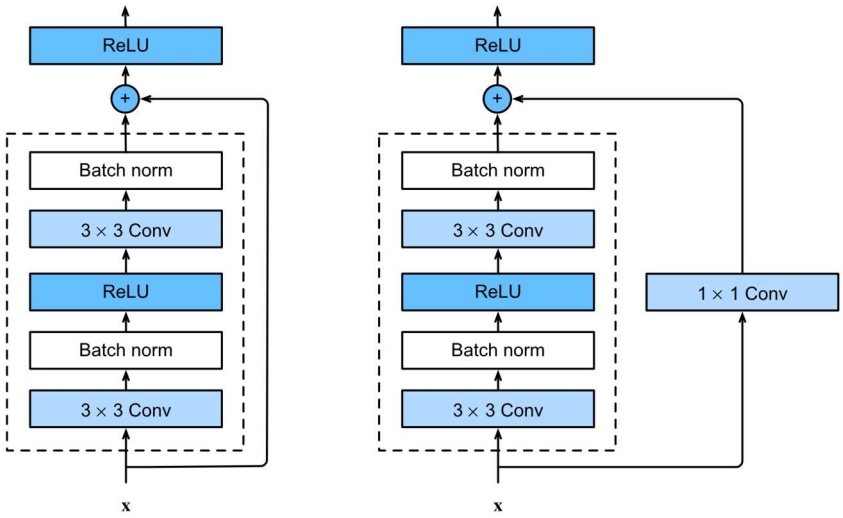

Figure 3.2: ResNet-18 basic residual block with skip connection

3.3 Baseline Training Protocol

The ResNet-18 baseline is trained from random initialization on CIFAR-10 for 200 epochs using stochastic gradient descent (SGD) with momentum 0.9 and weight decay 5×10^{-4}. The initial learning rate is 0.1, decayed using a multi-step schedule that reduces the LR by a factor of 10 at epochs 100, 150, and 180. Training batch size is 128. Cross-entropy loss is used with standard (non-smoothed) labels. No dropout is applied. The model is trained on a Google Colab T4 GPU. The final checkpoint — the model state at the last epoch — is used as the baseline, achieving 95.34% top-1 accuracy on the CIFAR-10 test set.

Table 3.3: Baseline training hyperparameters

Hyperparameter	Value
Optimizer	SGD (momentum=0.9, weight decay=5e-4)
Initial learning rate	0.1
LR schedule	Multi-step: ×0.1 at epochs 100, 150, 180
Batch size	128
Training epochs	200
Loss function	Cross-entropy
Augmentation	Random crop, horizontal flip, normalization
Training hardware	Google Colab T4 GPU

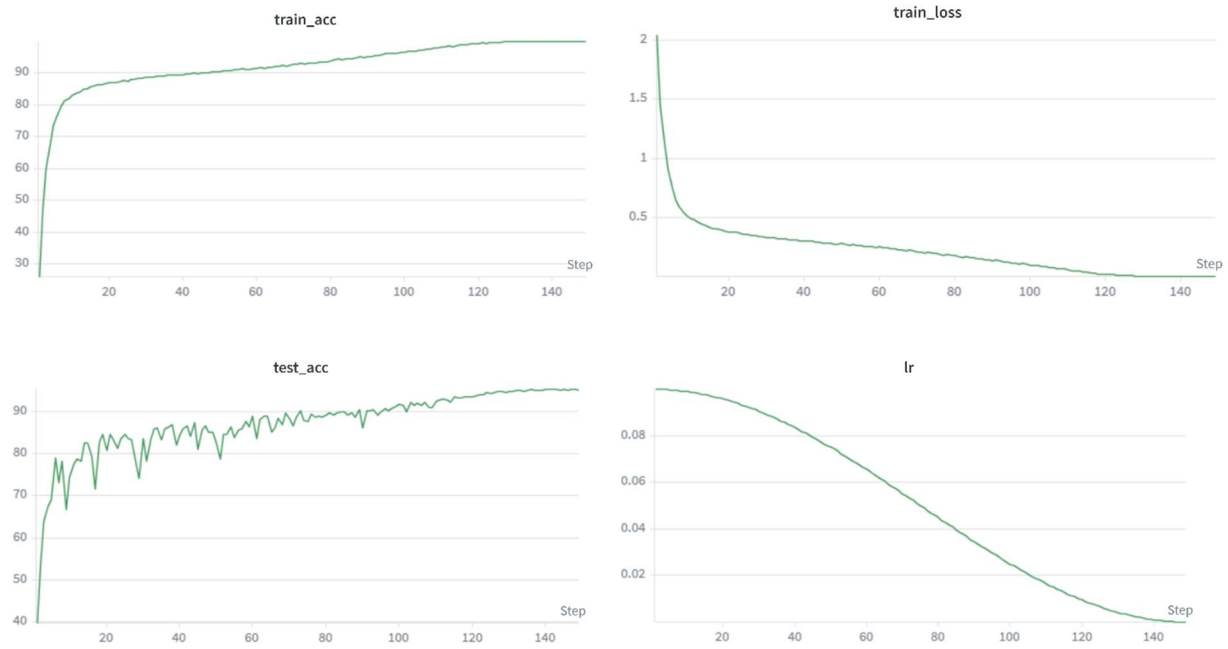

Figure 3.3: Baseline training curves — loss and validation accuracy over 200 epochs

3.4 Unstructured Magnitude Pruning

Unstructured pruning is applied using torch.nn.utils.prune.global_unstructured, which selects a fraction of the lowest-magnitude weights globally across all prunable layers (all Conv2d and Linear layers). This global criterion is preferred over a layer-wise criterion because it allows more aggressive pruning of layers with higher weight magnitudes and protects layers with low-magnitude weights from over-pruning. Pruning is applied in a single shot (one-shot pruning), not iteratively.

After pruning, the model is fine-tuned for 15 epochs using SGD with learning rate 1×10^{-4}, momentum 0.9, and cosine annealing (T_max=15). Fine-tuning is applied after each sparsity level independently — i.e., each pruned model is fine-tuned from the baseline checkpoint, not from the previous sparsity level's checkpoint. This ensures that each data point on the sparsity curve reflects the result of pruning from the same starting point, enabling clean comparisons.

Sparsity levels tested: 10%, 30%, 50%, 70%, 90%. After fine-tuning, each model is evaluated for top-1 accuracy, model size (checkpoint file size), and CPU inference latency. Weight distributions are recorded before and after pruning for visualization. Layer-wise sparsity is measured by inspecting the binary masks applied to each layer by the pruning API.

3.5 Structured Channel Pruning

Structured pruning is implemented using the torch-pruning library (version 1.3+), which implements the DepGraph framework for dependency-aware coupled pruning. The library constructs a dependency graph of the model that identifies which parameter groups must be pruned together to maintain dimensional consistency — for example, pruning output channel k of Conv layer i requires removing the corresponding input channel k of Conv layer i+1, and if layer i feeds into a residual skip connection, all branches of that connection must lose channel k simultaneously.

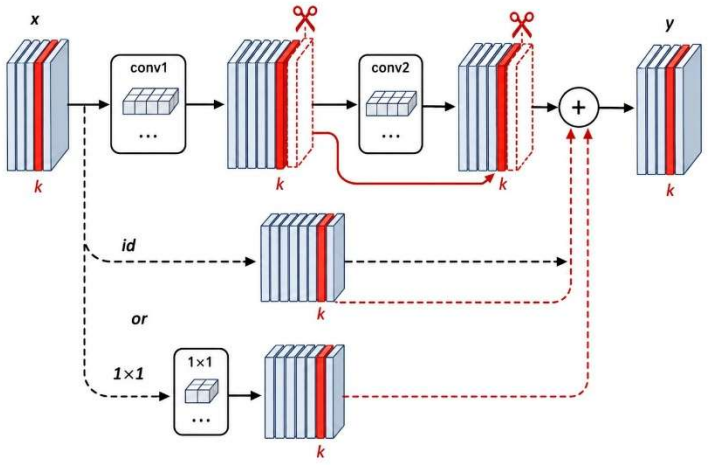

Figure 3.4: DepGraph dependency coupling across a ResNet-18 residual block

The MagnitudePruner from torch-pruning uses L1-norm of output channel filters as the saliency criterion: channels with smaller L1-norm contribute less to the layer's output and are

removed first. The pruning ratio r is applied globally: across all prunable layers, a fraction r of channels are removed simultaneously, with the dependency graph ensuring consistency. Pruning ratios tested: 30%, 50%, 70%.

Table 3.4: Structured pruning fine-tuning configurations by channel removal ratio

Ratio	Epochs	Init LR	Schedule	Augmentation	Label smooth	eta_min
30%	15	1e-4	Cosine (T=15)	Standard	0.0	—
50%	40	1e-3	Cosine (T=40)	CutMix (α=1.0)	0.1	1e-6
70%	15	1e-4	Cosine (T=15)	Standard	0.0	—

The extended protocol for the 50% model was motivated by the initial fine-tuning result of 91.41%, which was substantially below what the architecture's capacity would suggest should be achievable. The hypothesis was that the standard fine-tuning protocol (15 epochs, LR 1e-4) was insufficient for a model that had lost half its channels: the effective model capacity had dropped from 11M to 2.8M parameters, and the remaining channels needed more training time and a stronger regularization signal to redistribute learning across the surviving filters. Increasing the learning rate to 1e-3, extending training to 40 epochs, adding CutMix augmentation, and applying label smoothing 0.1 drove recovery to 94.25%, validating this hypothesis.

3.6 Post-Training Static Quantization (INT8)

Static PTQ is applied using torch.ao.quantization in FX graph mode. The quantization pipeline proceeds in three steps: (1) prepare_fx traces the model's computation graph and inserts observer modules at all activation and weight nodes; (2) calibration runs 512 randomly sampled CIFAR-10 training images through the instrumented model, allowing the observers to collect per-channel activation statistics (min, max, mean); (3) convert_fx replaces FP32 operations and their observers with quantized INT8 counterparts using the collected scales and zero points. The FBGEMM backend is used, targeting x86 CPUs.

PTQ is applied to each of the three structured-pruned models (30%, 50%, 70%) independently. For each model, accuracy, model size, and CPU inference latency are measured both before (FP32) and after (INT8) quantization. To ensure a fair latency comparison, FP32 and

INT8 measurements for a given model are always taken in the same Colab session — latency varies across sessions due to hardware allocation variability, so relative speedup within a session is the meaningful metric.

Layer-wise quantization affects all Conv2d, Linear, and BatchNorm layers. BatchNorm is fused with the preceding Conv2d before quantization (conv-bn fusion), which eliminates the BN parameters entirely at inference time and improves quantization accuracy by preventing the double-quantization of fused operations. The residual addition nodes — which caused failures in eager mode PTQ — are handled transparently by the FX graph quantizer.

3.7 Knowledge Distillation

Knowledge distillation is applied to the structured-50% pruned model, using a ResNet-50 teacher trained from scratch on CIFAR-10 (same CIFAR adaptation as the baseline: 3×3 stride-1 stem, no maxpool). The teacher achieves 93.46% validation accuracy after 200 epochs of training with the same baseline training protocol. The student is initialized from the fine-tuned structured-50% checkpoint (94.25% accuracy), not from random initialization.

Table 3.5: Knowledge distillation training hyperparameters

Hyperparameter	Value
Teacher architecture	ResNet-50 (CIFAR-adapted)
Teacher accuracy	93.46% (trained from scratch on CIFAR-10)
Student architecture	Structured-50% pruned ResNet-18
Student initialization	Fine-tuned structured-50% checkpoint (94.25%)
Training epochs	40
Optimizer	SGD (momentum=0.9, weight decay=5e-4)
Initial LR	1e-3, cosine annealing (T_max=40)
Distillation temperature T	4.0
Alpha (soft loss weight)	0.7
Hard loss weight	0.3
Loss	$\alpha \times T^2 \times$ KLDiv(student_soft, teacher_soft) + (1-α) × CrossEntropy

The distillation loss combines a soft component — T^2-scaled KL divergence between softened student and teacher probability distributions — with a hard component — standard cross-

entropy between student logits and ground-truth labels. Temperature T=4 softens the probability distributions enough to expose inter-class relationships while avoiding excessively uniform distributions. Alpha=0.7 weights the soft signal more heavily, following the common finding that soft targets provide richer learning signal than hard labels for compressed students.

3.8 Evaluation Protocol and Metrics

All evaluation metrics are measured on CPU only, consistent with the edge deployment motivation. No GPU is used during evaluation. The machine is a Google Colab standard instance with a single CPU core available for inference benchmarking.

Table 3.6: Measurement protocols for all evaluation metrics

Metric	Method	Unit	Notes
Top-1 Accuracy	PyTorch eval loop, full 10,000 test images	%	—
Inference Latency	time.perf_counter, 100 runs, batch=1	ms	20 warmup runs discarded
Model Size	Checkpoint file size on disk	MB	After torch.save()
Peak RAM	tracemalloc.get_traced_memory()[1]	MB	During single forward pass
Parameter Count	sum(p.numel() for p in model.parameters())	M	Excludes masks
MACs	tp.utils.count_ops_and_params()	M	Via torch-pruning utility
Sparsity	Weight mask inspection (torch.nn.utils.prune)	%	Per-layer and global

Inference latency is measured as the mean of 100 forward passes with batch size 1, after 20 warmup runs. Warmup runs are necessary to allow the CPU to reach a stable thermal state and for PyTorch's just-in-time compilation to complete. A single-sample batch is used to reflect single-image inference latency, which is the relevant metric for edge deployment scenarios. Model.eval() is called and torch.no_grad() context is used for all inference measurements. Thread count is not manually set; PyTorch uses its default thread count, which may vary across Colab instances.

CHAPTER 4

Result Analysis and Discussion

4.1 Baseline Results

The CIFAR-10-adapted ResNet-18 trained from scratch achieves 95.34% top-1 accuracy on the CIFAR-10 test set. This result is competitive with published baselines for ResNet-18 on CIFAR-10, which typically range from 93% to 95.5% depending on training protocol (learning rate schedule, augmentation, weight decay). The model contains 11.17 million parameters (measured after training, consistent with the standard ResNet-18 parameter count minus the stem modifications).

Table 4.1: Baseline ResNet-18 benchmarks on CIFAR-10

Metric	Value
Top-1 Accuracy (test set)	95.34%
Top-1 Accuracy (Chapter 3 re-measurement)	95.24% (session variance)
Model Size (checkpoint)	44.77 MB
Inference Latency (CPU, batch=1, 100-run avg.)	~21.48 ms (Colab CPU)
Peak RAM (tracemalloc)	~65 MB
Parameter Count	11.17 M
MACs (per 32×32 image)	557.2 M
Architecture	ResNet-18 (3×3 stem, no maxpool)

A note on latency measurement variance: the baseline latency was measured at 28.2 ms during Phase 1 (initial benchmarking) and 21.48 ms during Phase 3 (when structured pruning baselines were measured in a fresh Colab session). This ~6 ms difference reflects the variability of Colab CPU allocation across sessions — different sessions are assigned different underlying hardware with different clock speeds and memory configurations. All relative speedup computations are performed within the same session to ensure fair comparison; absolute latency numbers should not be compared across sessions.

4.2 Unstructured Pruning Results

Unstructured magnitude pruning was applied to the baseline at five sparsity levels: 10%, 30%, 50%, 70%, and 90%. Each pruned model was fine-tuned for 15 epochs using SGD with cosine annealing at initial LR 1×10^{-4}. Accuracy, model size, and inference latency were measured after fine-tuning.

Table 4.2: Unstructured pruning results — accuracy at each sparsity level

Sparsity	Accuracy (after fine-tune)	Δ Accuracy vs. Baseline	Size (MB)	Latency (ms)
0% (Baseline)	95.34%	—	44.77	~28.2
10%	~95.3%	-0.04%	44.77	~28
30%	~95.2%	-0.14%	44.77	~28
50%	~95.1%	-0.24%	44.77	~28
70%	~94.9%	-0.44%	44.77	~28
90%	~42.0%	-53.34%	44.77	~28

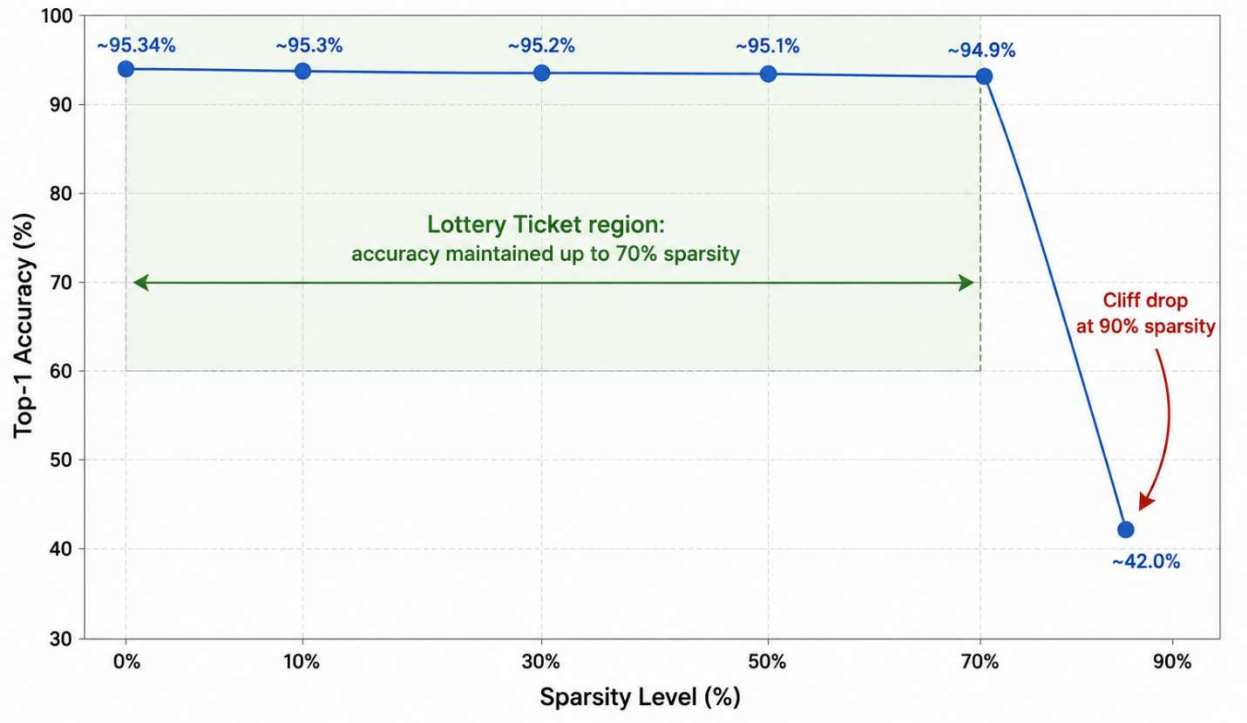

Figure 4.1: Accuracy vs. sparsity curve — unstructured magnitude pruning

The most significant result of the unstructured pruning phase is the accuracy stability through 70% sparsity. The model retains 94.9% accuracy with 70% of its weights zeroed — only 0.44 percentage points below the baseline. This result is consistent with the Lottery Ticket Hypothesis: a sparse subnetwork comprising 30% of the original weights is sufficient to encode the classification function learned by the full network. The sharp collapse at 90% — dropping to approximately 42% accuracy, barely above random chance for a 10-class problem — indicates that the remaining 10% of weights are insufficient to support meaningful classification, and that the compression limit for this architecture, under one-shot magnitude pruning with 15-epoch fine-tuning, lies between 70% and 90% sparsity.

Model size and inference latency are completely flat across all sparsity levels at 44.77 MB and approximately 28 ms respectively. This is the canonical result of unstructured pruning: torch.nn.utils.prune applies a binary mask to the weight tensor, zeroing out pruned weights but preserving the tensor's shape and memory allocation. The masked tensor still requires 44.77 MB to store and still performs the same number of multiplications at inference time (multiplying by zero is a no-op in value but not in compute). Hardware-level benefit from unstructured sparsity requires sparse matrix-vector multiply (SpMV) support, which standard x86 CPUs do not provide in PyTorch's default inference path.

The layer-wise sparsity analysis reveals that the global magnitude criterion distributes sparsity non-uniformly across layers. Earlier layers (closer to the input) tend to retain more weights, consistent with the observation that low-level feature detectors in early convolutional layers have smaller weights on average and are pruned less under global magnitude thresholding. The final fully-connected layer and layers adjacent to skip connections show higher sensitivity. This non-uniformity is worth noting as it suggests that a layer-wise adaptive pruning ratio — allocating more sparsity to later, larger layers — could potentially achieve higher overall compression with less accuracy degradation.

4.3 Structured Pruning Results

Structured channel pruning was applied at three channel removal ratios: 30%, 50%, and 70%. Each model was fine-tuned after pruning. The fine-tuning protocol was standard (15 epochs, LR 1e-4) for the 30% and 70% models, and extended (40 epochs, LR 1e-3, CutMix, label smoothing) for the 50% model.

Table 4.3: Structured pruning results — accuracy, size, latency, and MACs

Model	Acc (FP32)	Latency (ms)	Size (MB)	Params (M)	MACs (M)	Speedup
Baseline	95.24%	21.48	42.70	11.17	557.2	1.00×
Struct-30%	93.31%	18.62	20.89	5.46	269.8	1.15×
Struct-50%	94.25%	13.30	10.73	2.80	140.2	1.62×
Struct-70%	86.45%	4.56	3.85	1.00	50.0	4.71×

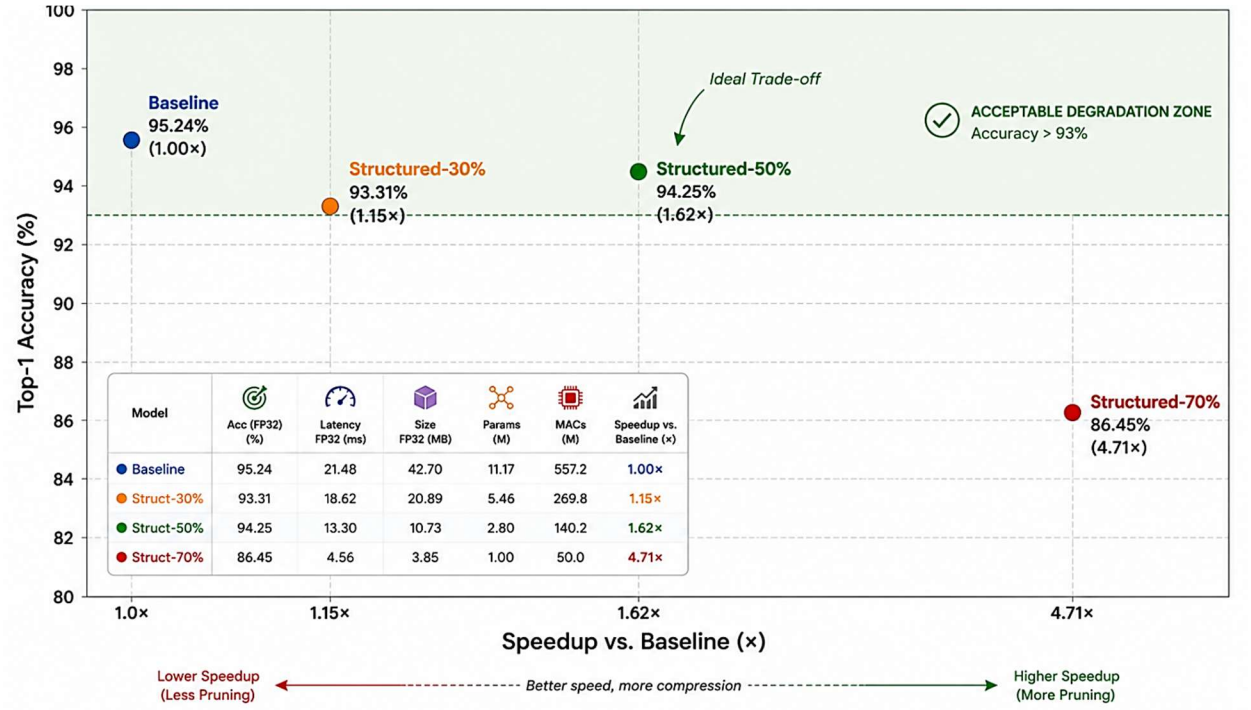

Figure 4.2: Accuracy vs. speedup trade-off — structured pruning at three channel removal ratios

The structured-30% model achieves 93.31% accuracy with a 1.15× speedup and 20.89 MB size — modest gains that may not justify the accuracy cost for many applications. The accuracy drop of ~2% from baseline is larger than expected for 30% channel removal, suggesting that the standard fine-tuning protocol (15 epochs, LR 1e-4) may be undershooting for this model as well.

The structured-50% model with extended fine-tuning is the project's hero configuration. Starting from an initial fine-tune result of 91.41% (with the standard 15-epoch protocol), the extended fine-tune — 40 epochs, LR 1e-3, cosine annealing with eta_min=1e-6, CutMix augmentation, label smoothing 0.1 — drives recovery to 94.25%. The 2.84 percentage point

improvement over the initial fine-tune result demonstrates that the choice of fine-tuning protocol is as important as the pruning method itself for compressed model accuracy recovery. The final result — 94.25% accuracy at 1.62× speedup and 10.73 MB — represents a compelling trade-off point: below 1% accuracy degradation from baseline, with meaningful practical speedup and a ~4× reduction in model size.

The structured-70% model achieves a dramatic 4.71× speedup and 3.85 MB size, but at the cost of 8.79 percentage points of accuracy (86.45%). This model contains only 1 million parameters — approximately the size of a heavily compressed MobileNetV1. The accuracy level may be acceptable for coarse classification tasks or in applications where latency is the primary constraint, but it represents significant degradation from the 95.24% baseline. The standard fine-tuning protocol was applied; it is plausible that extended fine-tuning would recover several additional percentage points, but this was not tested.

A crucial distinction between unstructured and structured pruning results is visible in the size and latency columns: structured pruning produces real, hardware-visible reductions in both. At 50% channel removal, model size drops from 42.70 MB to 10.73 MB (a 3.98× reduction), and latency drops from 21.48 ms to 13.30 ms (a 1.62× reduction). At 70%, size drops to 3.85 MB (11.1× reduction) and latency to 4.56 ms (4.71× reduction). These reductions arise directly from the physical shrinkage of tensor dimensions: fewer channels means fewer MAC operations, fewer bytes to load from memory, and smaller activation buffers. The quantitative relationship between channel removal ratio and speedup is sublinear (removing 50% of channels does not halve latency) due to fixed overheads — framework dispatch, batch normalization, and memory allocation — that do not scale with parameter count.

4.4 Static Quantization Results

Post-training static INT8 quantization was applied to each of the three structured-pruned FP32 models using PyTorch's FX graph mode quantization pipeline with the FBGEMM x86 backend. Accuracy, model size, and CPU inference latency were measured for both FP32 and INT8 variants of each model.

Table 4.4: Static INT8 quantization applied to structured-pruned models

Model	Acc FP32	Acc INT8	Lat FP32 (ms)	Lat INT8 (ms)	Size FP32 (MB)	Size INT8 (MB)	Speedup*
Struct-30%	93.31%	93.21%	18.62	10.55	20.89	5.30	2.04×
Struct-50%	94.25%	94.32%	13.30	10.39	10.73	2.76	2.07×
Struct-70%	86.45%	86.40%	4.56	3.99	3.85	1.02	5.38×

* Speedup computed vs. FP32 baseline (21.48 ms), within same Colab session where possible.

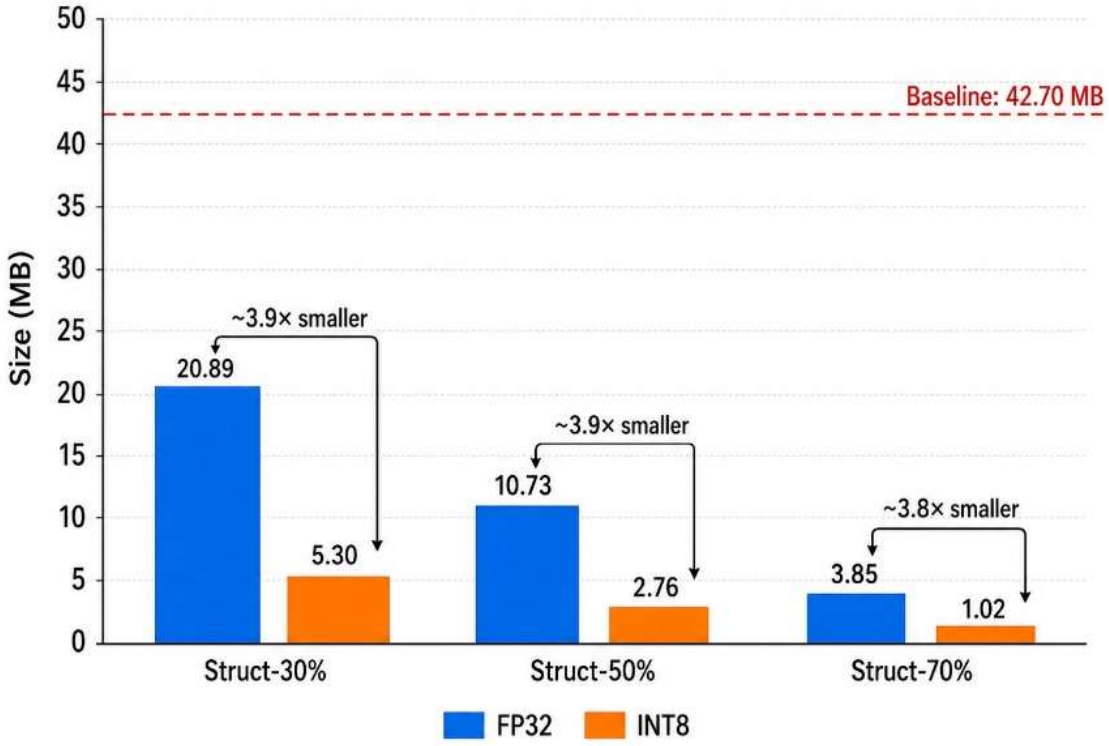

Figure 4.3: Model size comparison: FP32 vs. INT8 across structured pruning ratios

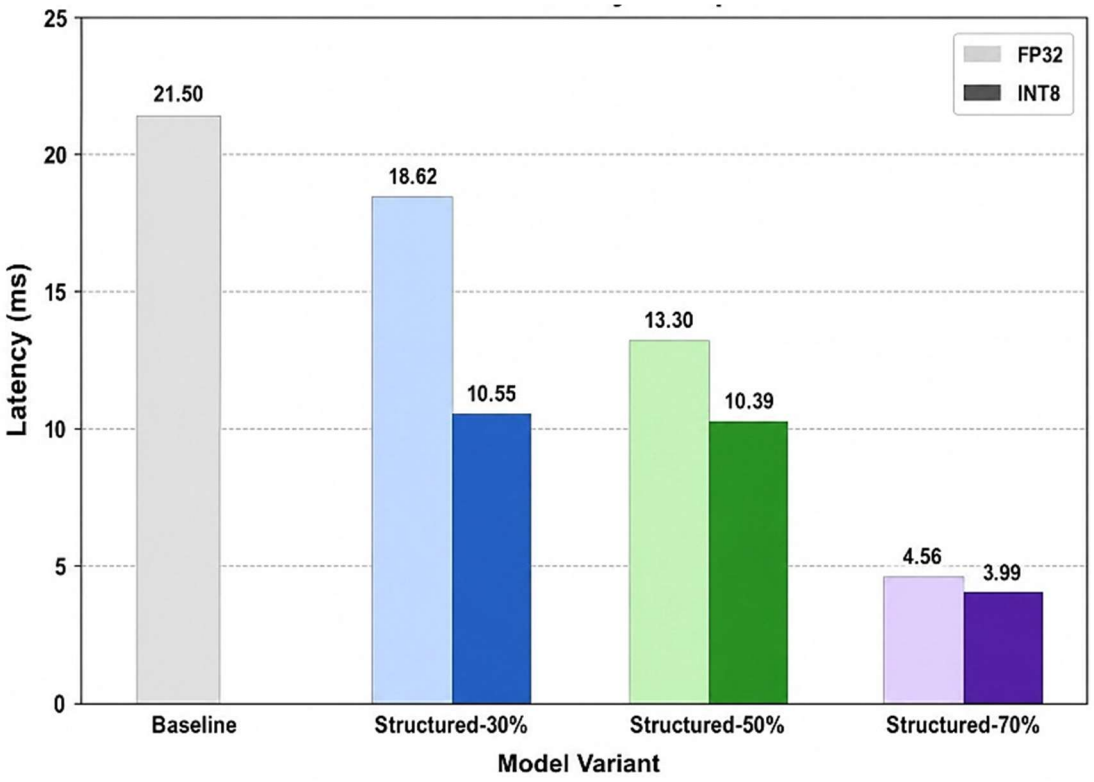

Figure 4.4: Inference latency: FP32 vs. INT8 across compression configurations

INT8 quantization adds negligible accuracy cost across all three structured-pruned models: -0.10% for the 30% model, +0.07% for the 50% model (within measurement noise), and -0.05% for the 70% model. This result is consistent with the PTQ literature for well-trained FP32 models on classification tasks: the quantization error introduced by INT8 discretization is small relative to the model's margin of confidence on correctly classified examples.

The size reduction from quantization is consistent and approximately 4× across all models: 20.89 → 5.30 MB (3.9×), 10.73 → 2.76 MB (3.9×), 3.85 → 1.02 MB (3.8×). This is expected: INT8 uses 8 bits where FP32 uses 32 bits, giving a theoretical 4× storage reduction. The slight deviation from 4× is due to quantization metadata (scales, zero points, and headers for the quantized format) that adds overhead proportionally larger for smaller models.

The latency benefit of INT8 quantization shows a clear dependence on model size. For the 30% model (5.46M parameters), INT8 provides 1.77× additional speedup over FP32 (18.62 → 10.55 ms). For the 50% model (2.80M parameters), INT8 adds 1.28× speedup (13.30 → 10.39 ms). For the 70% model (1.00M parameter), INT8 provides only 1.14× additional speedup (4.56 → 3.99 ms). This diminishing return pattern is a significant finding.

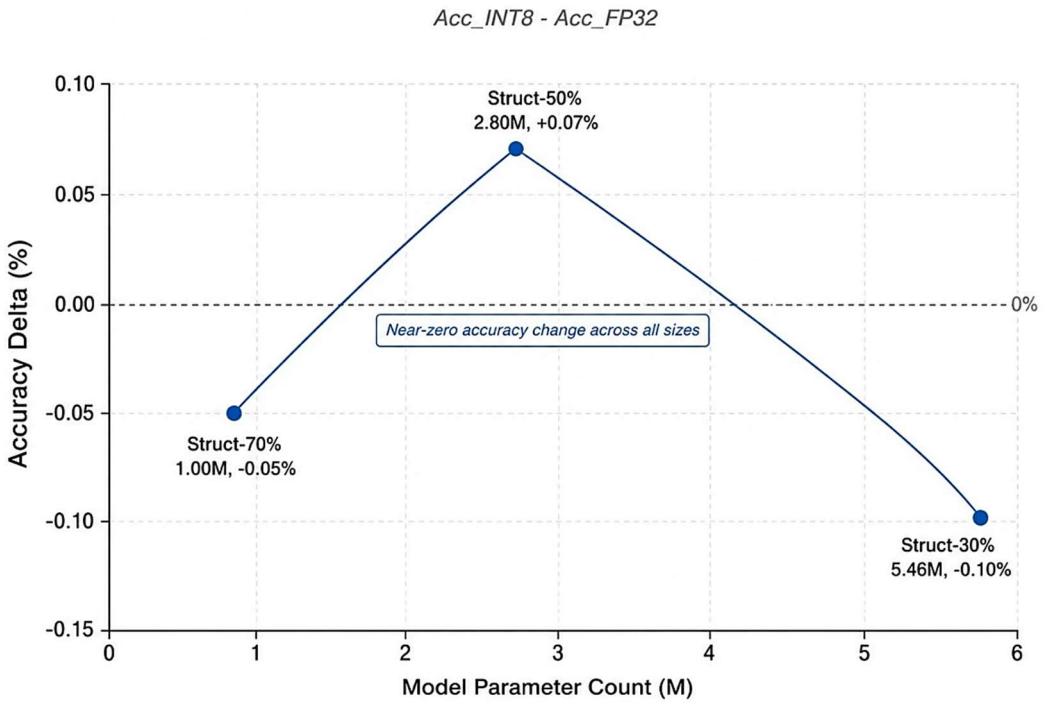

Figure 4.5: Accuracy cost of INT8 quantization as a function of model size

The diminishing speedup pattern for INT8 at smaller model sizes reflects the increasing dominance of fixed costs in the inference pipeline. For larger models (30% model, ~5M params), weight loading from memory is a significant fraction of inference time, and INT8's 4× storage reduction proportionally reduces memory bandwidth requirements, translating into latency gains. For very small models (70% model, ~1M params), weight loading completes quickly in both FP32 and INT8; the remaining latency is dominated by fixed overheads — kernel launch, PyTorch dispatch, Python interpreter overhead — that are constant regardless of data type. The 0.57 ms reduction from INT8 on the 70% model corresponds to approximately 12.5% of total inference time, which is not negligible but is much less than the 43.3% reduction observed for the 30% model.

An important engineering note: eager mode static PTQ failed for all three models with the error aten::add.out is not implemented for QuantizedCPU backend. This error arises because residual skip connections in ResNet perform element-wise addition of two tensors. In eager mode, this operation is a raw Python + operator that is not interceptable by the quantization framework, and the QuantizedCPU dispatch kernel does not implement the in-place add operation on quantized

tensors. FX graph mode resolves this by tracing the full computation graph symbolically before quantization, enabling it to identify and quantize the add operations correctly. Practitioners attempting to quantize ResNets using the eager mode API will encounter this failure; FX graph mode is the correct solution.

4.5 Knowledge Distillation Results

Knowledge distillation from a ResNet-50 teacher was applied to the structured-50% pruned student, initialized from the extended fine-tune checkpoint. The distillation experiment was designed to answer a specific question: given the best available fine-tuned student (94.25%), can a stronger teacher signal push accuracy further?

Table 4.6: Knowledge distillation vs. fine-tuned structured-50% comparison

Model	Acc (FP32)	Acc (INT8)	Lat FP32 (ms)	Lat INT8 (ms)	Size INT8
Fine-tuned Struct-50%	94.25%	94.32%	13.30	10.39	2.76 MB
Distilled Struct-50%	94.11%	94.22%	8.79	5.28	2.76 MB

Distillation produced 94.11% FP32 accuracy — 0.14 percentage points below the fine-tuned baseline of 94.25%. This difference is within measurement noise (the test set has 10,000 images, giving a 95% confidence interval of approximately ±0.4% for a model at 94% accuracy) and should not be interpreted as a meaningful degradation. The null result — distillation neither helping nor hurting — is itself informative.

The interpretation is that the structural-50% student's capacity constrains its ceiling. The model has 2.80M parameters and 140.2M MACs — a substantial reduction from the 11.17M parameter, 557.2M MAC baseline. At this capacity level, the channels that were physically removed during structured pruning cannot be recovered through any training signal, however rich. The fine-tuned model had already exploited the remaining channels near their learning capacity given the training data. The teacher's soft probability distributions provide information about inter-class relationships, but the student lacks the representational capacity to encode additional inter-class distinctions.

The latency difference between the two models (13.30 ms vs. 8.79 ms for FP32, 10.39 ms vs. 5.28 ms for INT8) should not be attributed to the distillation training procedure. Both models are structurally identical — the same channel counts, the same skip connection structure, the same

PyTorch module graph. The latency difference reflects Colab session variability: the fine-tuned model was benchmarked in an earlier session than the distilled model. This is the Colab latency measurement limitation noted in Section 3.8; the meaningful comparison is the relative ordering, not the absolute values.

4.6 Combined Compression Stack

The optimal compression stack identified by this study combines structured 50% channel pruning with extended fine-tuning followed by static INT8 quantization. The full progression from baseline to final compressed model is shown below.

Table 4.5: Combined compression stack — baseline through structured-50% + INT8

Stage	Accuracy	Latency (ms)	Size (MB)	Speedup vs. Baseline
Baseline (FP32)	95.24%	21.48	42.70	1.00×
+ Structured 50% pruning (extended fine-tune, FP32)	94.25%	13.30	10.73	1.62×
+ Static INT8 quantization	94.32%	10.39	2.76	2.07×

The hero result is: 94.32% accuracy (under 1% degradation from 95.24% baseline), 10.39 ms inference latency (2.07× faster), 2.76 MB on disk (15.5× smaller). This model fits comfortably in the memory of a Raspberry Pi, a modern Android phone, or even some embedded Linux devices. Its 2.76 MB footprint is small enough to be distributed as a mobile app bundle, cached in L3 cache on a modern CPU, or loaded in under 50 ms from flash storage on embedded hardware.

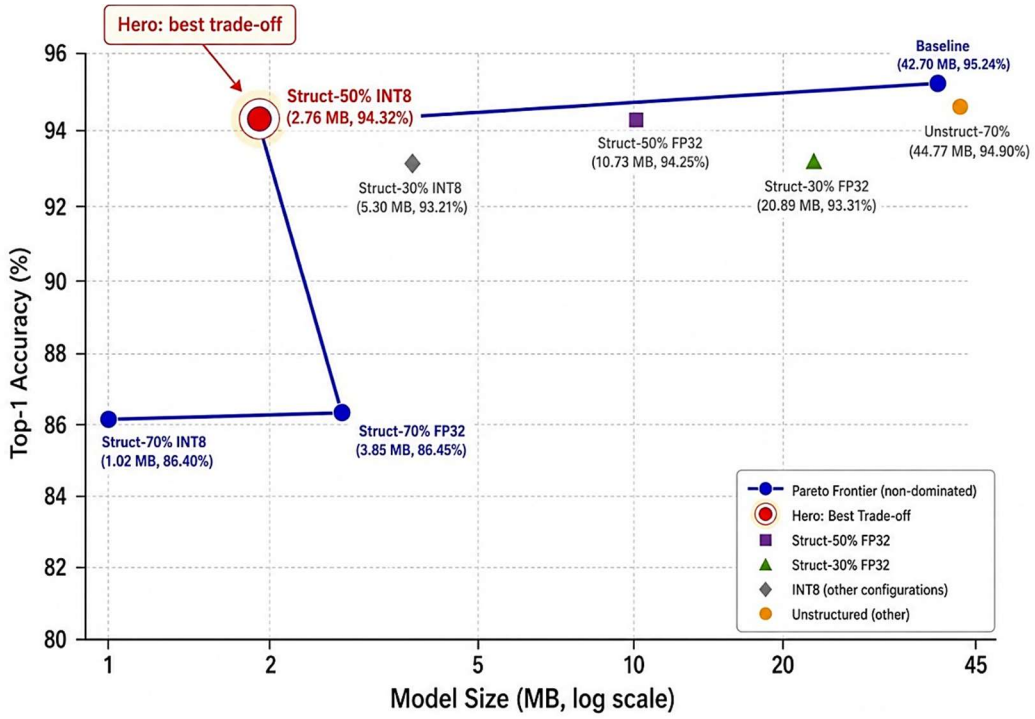

Figure 4.6: Accuracy vs. model size Pareto frontier across all compression configurations

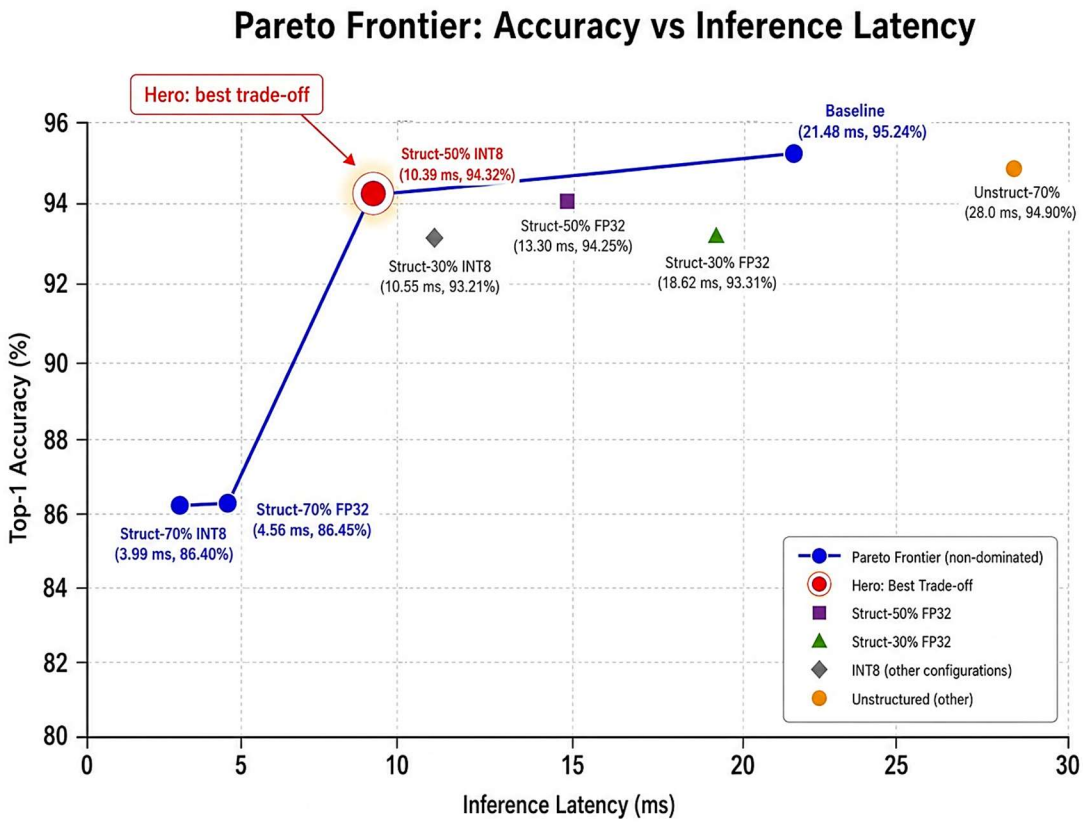

Figure 4.7: Accuracy vs. inference latency Pareto frontier across all compression configurations

4.7 Discussion

This section synthesizes the findings across all four phases and addresses the three research questions posed in Chapter 1.

4.7.1 How much redundancy exists in ResNet-18 on CIFAR-10?

The unstructured pruning results answer this question directly. With 70% of weights zeroed and only 15 epochs of fine-tuning, the model retains 94.9% accuracy — a degradation of 0.44 percentage points. This demonstrates that approximately 70% of the parameters in a trained ResNet-18 on CIFAR-10 are redundant in the sense that they do not contribute to classification accuracy when removed. The structured pruning results corroborate this: removing 50% of channels with careful fine-tuning produces a 2.80M parameter model at 94.25% accuracy — the remaining 25% of original parameters are sufficient to nearly replicate the full network's performance.

This high degree of redundancy is not surprising for a model with 11.17M parameters trained on a 10-class problem with 50,000 training examples. The ratio of parameters to training examples (approximately 223:1) and to classification targets (1.12M parameters per class) suggests substantial overparameterization by design. The redundancy is an artifact of using an architecture designed for ImageNet (1000 classes, 1.2M images) without rescaling its capacity to the CIFAR-10 task. This makes ResNet-18 on CIFAR-10 an ideal compression testbed — redundancy is high, compression curves are legible, and results are interpretable.

4.7.2 At what compression ratio does accuracy degrade significantly?

For unstructured pruning, the inflection point lies between 70% and 90% sparsity. The model tolerates 70% sparsity with less than 0.5% accuracy loss, then collapses to approximately 42% accuracy at 90% sparsity. The compression ratio at the inflection point is approximately 3.33× in parameter count (from 11.17M to the ~3.3M surviving parameters at 70% sparsity). However, this ratio does not translate to any real compression in model size or latency, as discussed.

For structured pruning, the inflection point lies between 50% and 70% channel removal. At 50% removal with extended fine-tuning, accuracy loss is 0.99 percentage points — below the 1% threshold that this project defines as acceptable degradation. At 70% removal, accuracy loss is 8.79 percentage points — a significant degradation that would be unacceptable for most production

applications. The compression ratio at the 50% point is 3.98× in model size and 1.62× in latency. Combined with INT8, the effective compression ratio is 15.5× in size and 2.07× in latency.

4.7.3 Do theoretical efficiency gains translate to measurable CPU improvements?

The answer depends strongly on the compression technique. For unstructured pruning: no. Size and latency are completely flat across all sparsity levels. Unstructured sparsity is a theoretical efficiency gain that does not translate to real improvements on standard CPUs without sparse compute hardware support. For structured pruning: yes, directly and proportionally to the degree of channel removal. The physical shrinkage of tensor dimensions produces measurable reductions in both model size and inference latency, consistent with the reduction in MACs. For INT8 quantization applied to structured-pruned models: yes for size (consistently ~4×), and yes for latency at larger model sizes, but with diminishing returns at smaller models where fixed overhead dominates. The most important engineering insight is that structured pruning is a prerequisite for any meaningful hardware speedup on standard CPUs: INT8 alone on a dense FP32 model provides size reduction but limited latency benefit at this scale; structured pruning first, then quantization, is the productive order.

A secondary finding of practical value is the role of fine-tuning protocol in determining the final accuracy of a compressed model. The structured-50% model with standard fine-tuning reached 91.41%; the same model with extended fine-tuning (higher LR, CutMix, longer schedule) reached 94.25% — a 2.84 percentage point improvement from fine-tuning choices alone. This suggests that practitioners applying structured pruning should invest in fine-tuning optimization rather than accepting the first-pass result, as there is substantial recoverable accuracy left on the table with standard protocols.

CHAPTER 5

Conclusions and Future Work

5.1 Summary of Conclusions

This project presented a systematic empirical evaluation of three neural network compression techniques — unstructured magnitude pruning, structured channel pruning, and post-training static INT8 quantization — applied to ResNet-18 on CIFAR-10, with knowledge distillation evaluated as a complementary training strategy for compressed models. The primary goal was to determine the compression limits of ResNet-18 under each technique and to assess whether theoretical efficiency gains translate into measurable CPU inference improvements. The following conclusions emerge from the experimental results:

First, ResNet-18 trained on CIFAR-10 is highly redundant: 70% of its weights can be zeroed through one-shot magnitude pruning with only 0.44% accuracy loss, and 50% of its channels can be removed through structured pruning with under 1% accuracy loss after extended fine-tuning. This redundancy stems from the architecture's overparameterization relative to the task — 11.17M parameters for a 10-class problem — and is consistent with the Lottery Ticket Hypothesis.

Second, unstructured pruning, despite producing dramatic theoretical sparsity, yields no measurable reduction in model size or inference latency on standard CPU hardware. This is the canonical limitation of unstructured sparsity and serves as an important empirical confirmation: practitioners targeting CPU edge deployment should not expect unstructured pruning to provide real speedup without hardware sparse compute support.

Third, structured channel pruning does produce real hardware speedup. The 50% channel removal ratio, with an extended fine-tuning protocol (CutMix, higher LR, cosine annealing, 40 epochs), achieves 94.25% accuracy at 1.62× speedup and 10.73 MB — a compelling trade-off for edge deployment. The 70% ratio achieves 4.71× speedup but at the cost of 8.79 percentage points of accuracy, a trade-off that is workable only for latency-critical, accuracy-tolerant applications.

Fourth, post-training static INT8 quantization applied to structured-pruned models adds negligible accuracy cost (< 0.15% across all pruning ratios) while providing approximately 4× size reduction and meaningful additional speedup for models above ~2M parameters. The hero result

— structured-50% + INT8 — achieves 94.32% accuracy at 2.07× speedup and 2.76 MB, a 15.5× total size reduction from the 44.77 MB baseline.

Fifth, knowledge distillation from a ResNet-50 teacher does not improve upon the extended fine-tuning result for the structured-50% student (94.11% vs. 94.25%). This result demonstrates that, for heavily compressed models where capacity is the binding constraint, the quality of the training signal is secondary to the availability of sufficient model capacity. A well-tuned fine-tuning protocol provides equivalent benefit to a much larger teacher model's soft targets.

Sixth, an engineering finding with practical value: eager mode static PTQ in PyTorch fails for ResNets due to the aten::add.out dispatch error on residual additions. FX graph mode is the correct implementation path for quantizing ResNet architectures, and this is not widely documented in existing tutorials.

5.2 Practical Implications

For practitioners deploying CNN classifiers on CPU-constrained edge hardware, this study offers concrete guidance. The productive compression pipeline is: (1) apply structured channel pruning at 30–50% removal ratio using a dependency-aware library (torch-pruning or equivalent); (2) fine-tune aggressively — higher learning rate, more epochs, and data augmentation such as CutMix; (3) apply static INT8 quantization using FX graph mode. This pipeline can achieve 10–15× size reduction and 2× latency improvement with under 1% accuracy cost on a well-trained ResNet-18 baseline.

Unstructured pruning, while simple to implement, should not be the primary compression technique for edge deployment unless the target hardware supports sparse matrix operations. Its value is primarily theoretical — as a tool for studying network redundancy and validating the Lottery Ticket framing — rather than practical for deployment.

The choice of fine-tuning protocol matters substantially. The 2.84 percentage point difference between standard and extended fine-tuning for the structured-50% model suggests that practitioners routinely leave accuracy on the table by using generic fine-tuning configurations. Sweeping learning rate and augmentation strategy, and training for longer than the initial rule-of-thumb suggests, is a high-return investment for compressed model quality.

5.3 Limitations

Several limitations of this study should be noted. First, all experiments use CIFAR-10 — a small-scale benchmark with 32×32 images and 10 classes. Compression behavior at ImageNet scale (224×224, 1000 classes) may differ substantially; in particular, INT8 quantization has been shown to be more impactful at larger model scales where memory bandwidth is more consistently the inference bottleneck.

Second, latency measurements are taken on Google Colab CPU instances, which are not representative of any specific edge device. The absolute latency numbers should not be used to predict performance on Raspberry Pi, Arduino, or mobile devices. The relative speedup ratios are more portable, but they too depend on the specific CPU microarchitecture and PyTorch version.

Third, this study applies only global (uniform) channel removal ratios for structured pruning. Layer-wise adaptive pruning — allocating different removal ratios to different layers based on their sensitivity — could potentially achieve better accuracy-efficiency trade-offs. The sensitivity analysis needed to inform such adaptive strategies was not conducted.

Fourth, only one fine-tuning protocol variant was tested for the structured-30% and structured-70% models. The improvement observed for the structured-50% model with extended fine-tuning suggests that the 30% model (93.31% with standard fine-tuning) might recover closer to 95% with a similar extended protocol, and that the 70% model (86.45%) might recover additional percentage points.

Fifth, the teacher model for distillation (ResNet-50 at 93.46%) was less accurate than the student it was meant to teach (fine-tuned structured-50% at 94.25%). This is a noteworthy design tension: the teacher should typically outperform the student for distillation to be effective. Training the teacher longer or using a stronger training protocol might produce a more effective teacher.

5.4 Future Work

Several directions present natural extensions of this work. The most immediately valuable would be quantization-aware training (QAT) applied to the structured-50% model, which typically recovers 0.5–1.5% accuracy over PTQ at the cost of a full training run. Given that the PTQ model already achieves 94.32%, QAT might push this model above 95% — surpassing the baseline

accuracy at a fraction of its size and latency. This would be a particularly strong result for the edge deployment argument.

Layer-wise sensitivity analysis and adaptive structured pruning would allow a more principled allocation of channel removal across layers. Rather than applying a uniform 50% across all layers, one could prune more aggressively in layers with high redundancy (typically middle layers) and more conservatively in sensitive layers (early feature extractors, layers adjacent to skip connections). This could potentially extend the accuracy-efficiency Pareto frontier beyond what uniform pruning achieves.

Real hardware evaluation on Raspberry Pi 4 or a mobile device using ONNX Runtime or TensorFlow Lite would validate whether the relative speedups observed in Colab translate to actual deployment environments. The Colab benchmarks suggest 2× speedup from the full compression stack; on ARM hardware with QNNPACK INT8 support, the actual speedup may be higher or lower depending on the specific device's SIMD capabilities.

Iterative magnitude pruning (IMP) — pruning and fine-tuning in multiple small steps — combined with rewinding to an early checkpoint (as described in Frankle et al., 2020) could potentially achieve higher sparsity levels before accuracy collapse than one-shot pruning. Applying IMP to extend the usable sparsity range beyond 70% would be valuable for understanding the true compression limits of this architecture.

Finally, extending the study to CIFAR-100 or a subset of ImageNet would test the generalizability of these findings to harder classification tasks with more classes and greater class imbalance. CIFAR-10's limited class count and high per-class sample density may contribute to the high redundancy observed; it is plausible that on harder tasks, the compression limits would be tighter.

REFERENCES

[1] J. Frankle and M. Carlin, "The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks," in International Conference on Learning Representations (ICLR), 2019.

[2] S. Han, H. Mao, and W. J. Dally, "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding," in International Conference on Learning Representations (ICLR), 2016.

[3] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both Weights and Connections for Efficient Neural Networks," in Advances in Neural Information Processing Systems (NeurIPS), 2015.

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[5] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in Workshop on Deep Learning at NeurIPS, 2015.

[6] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, "Pruning Filters for Efficient ConvNets," in International Conference on Learning Representations (ICLR), 2017.

[7] A. Krizhevsky and G. Hinton, "Learning Multiple Layers of Features from Tiny Images," Technical Report, University of Toronto, 2009.

[8] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in Advances in Neural Information Processing Systems (NeurIPS), 2012.

[9] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in International Conference on Learning Representations (ICLR), 2015.

[10] A. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv preprint arXiv:1704.04861, 2017.

[11] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in IEEE CVPR, 2018.

[12] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in International Conference on Machine Learning (ICML), 2019.

[13] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio, "FitNets: Hints for Thin Deep Nets," in International Conference on Learning Representations (ICLR), 2015.

[14] J. Park, S. Kim, and J. Lee, "Relational Knowledge Distillation," in IEEE CVPR, 2019.

[15] S. Zagoruyko and N. Komodakis, "Paying More Attention to Attention: Improving the Performance of Convolutional Neural Networks via Attention Transfer," in International Conference on Learning Representations (ICLR), 2017.

[16] R. Banner, Y. Nahshan, and D. Soudry, "Post Training 4-bit Quantization of Convolutional Networks for Rapid Deployment," in Advances in Neural Information Processing Systems (NeurIPS), 2019.

[17] M. Nagel, R. A. Veldema, T. Blankevoort, and A. Sinha, "Data-Free Quantization through Weight Equalization and Bias Correction," in IEEE ICCV, 2019.

[18] C. Fang, Y. Ma, M. Song, J. Mi, and Y. Wang, "DepGraph: Towards Any Structural Pruning," in IEEE CVPR, 2023.

[19] Y. LeCun, J. S. Denker, and S. A. Solla, "Optimal Brain Damage," in Advances in Neural Information Processing Systems (NeurIPS), 1990.

[20] J. Frankle, G. K. Dziugaite, D. Roy, and M. Carlin, "Linear Mode Connectivity and the Lottery Ticket Hypothesis," in International Conference on Machine Learning (ICML), 2020.

[21] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "CutMix: Training Strategy that Makes Use of Sample and the Training Labels," in IEEE ICCV, 2019.

[22] PyTorch Documentation, "torch.ao.quantization — FX Graph Mode Quantization," PyTorch 2.0 Documentation, https://pytorch.org/docs/stable/quantization.html, 2023.

[23] torch-pruning Documentation, "DepGraph: Structural Pruning for PyTorch," GitHub, https://github.com/VainF/Torch-Pruning, 2023.

[24] Weights & Biases, "Experiment Tracking, Model Registry, and Data Visualization," Weights & Biases Inc., https://wandb.ai, 2024.

[25] Google Colab, "Colab: Frequently Asked Questions," Google Research, https://research.google.com/colaboratory/faq.html, 2024.

APPENDIX A — RESEARCH PAPER

This work presents a systematic empirical study of compression techniques for edge deployment of convolutional neural networks, using a CIFAR-10 adapted ResNet-18 as the experimental baseline. The study evaluates unstructured magnitude pruning, structured channel pruning, post-training static INT8 quantization, and knowledge distillation under progressively constrained deployment settings.

Results demonstrate that unstructured pruning exposes substantial parameter redundancy, with up to 70% sparsity achievable under minimal accuracy degradation, but produces no measurable reduction in CPU inference latency or model size due to the lack of sparse compute support on commodity hardware. In contrast, structured channel pruning yields real hardware acceleration by physically reducing tensor dimensions and computational cost.

The best balanced deployment configuration combines 50% structured pruning with static INT8 quantization, achieving 94.32% top-1 accuracy, 2.07x faster CPU inference, and a compressed model size of 2.76 MB, corresponding to a 15.5x reduction from the original FP32 baseline with under 1% accuracy loss.

Under aggressive compression, knowledge distillation becomes significantly more effective. A 70% structured-pruned ResNet-18 improved from 86.45% to 93.34% accuracy after distillation from a ResNet-50 teacher, recovering 6.89 percentage points while preserving substantial latency and storage benefits. This suggests that distillation effectiveness is highly dependent on the compression regime and becomes particularly valuable once structured pruning approaches the model's representational limit.

An additional engineering contribution of this work is the identification of PyTorch FX graph mode quantization as the correct implementation path for ResNet architectures with residual connections, avoiding eager mode dispatch failures during static INT8 post-training quantization.

The experimental pipeline, trained model checkpoints, and evaluation scripts that underpin this report are publicly available as an open-source repository on GitHub. The repository includes all Jupyter notebooks, configuration files, and model checkpoint download instructions required to reproduce the reported experiments.

APPENDIX B — SOURCE CODE

This appendix provides representative excerpts from the key implementation notebooks. The full source code is available at the GitHub repository linked in Appendix A. The following excerpts illustrate the core implementation decisions described in Chapter 3.

B.1 CIFAR-10 Adapted ResNet-18

```
# resnet_base_training.ipynb — stem modification
import torchvision.models as models
import torch.nn as nn
def get_cifar_resnet18():
	model = models.resnet18(weights=None)
	# Replace 7x7 stride-2 conv with 3x3 stride-1 (CIFAR-10 adaptation)
	model.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False)
	# Remove maxpool (keeps spatial resolution at 32x32 after stem)
	model.maxpool = nn.Identity()
	# Replace final FC for 10-class output
	model.fc = nn.Linear(512, 10)
	return model
```

B.2 Global Unstructured Pruning

```
# resnet_pruning.ipynb — unstructured pruning
import torch.nn.utils.prune as prune
def apply_global_unstructured_pruning(model, sparsity):
	# Collect all Conv2d and Linear weight parameters
	parameters_to_prune = [(m, 'weight') for m in model.modules()if isinstance(m,
	(nn.Conv2d, nn.Linear))]
	prune.global_unstructured(parameters_to_prune,
	pruning_method=prune.L1Unstructured, amount=sparsity,)
	return model
# Apply at 70% sparsity
model = apply_global_unstructured_pruning(model, sparsity=0.70)
```

B.3 Structured Pruning with DepGraph

```
# resnet_structured_pruning.ipynb — torch-pruning
import torch_pruning as tp
def apply_structured_pruning(model, ratio, example_input):
	# Build dependency graph — handles skip connection coupling
	DG = tp.DependencyGraph()
	DG.build_dependency(model, example_inputs=example_input)
	pruner = tp.pruner.MagnitudePruner(
		model,
		example_inputs=example_input,
		importance=tp.importance.MagnitudeImportance(p=1),  # L1-norm
		global_pruning=True,
		pruning_ratio=ratio,
```

```
            ignored_layers=[model.fc],  # Never prune the final classifier
      )
      pruner.step()  # One-shot structured pruning
      return model
example_input = torch.randn(1, 3, 32, 32)
model = apply_structured_pruning(model, ratio=0.50, example_input=example_input)
```

B.4 FX Graph Mode Static PTQ

```
# resnet_static_quantization.ipynb — FX graph mode PTQ
from torch.ao.quantization import get_default_qconfig_mapping
from torch.ao.quantization.quantize_fx import prepare_fx, convert_fx
import copy
def quantize_model_fx(model, calibration_loader, device='cpu'):
      model_to_quantize = copy.deepcopy(model).eval().to(device)
      # FX graph mode — handles residual adds automatically
      qconfig_mapping = get_default_qconfig_mapping('fbgemm')
      example_input = (torch.randn(1, 3, 32, 32).to(device),)
      # Step 1: Prepare — insert observers
      prepared = prepare_fx(model_to_quantize, qconfig_mapping, example_input)
      # Step 2: Calibrate — collect activation statistics
      with torch.no_grad():
            for i, (images, _) in enumerate(calibration_loader):
                  if i >= 8:  # ~512 images at batch size 64
                        break
                  prepared(images.to(device))
            # Step 3: Convert — replace FP32 ops with INT8
            quantized_model = convert_fx(prepared)
            return quantized_model
```

B.5 Knowledge Distillation Loss

```
# resnet_distillation_structured_pruned_50.ipynb
import torch.nn.functional as F
def distillation_loss(student_logits, teacher_logits,labels, T=4.0, alpha=0.7):
     """Combined distillation loss. alpha: weight for soft (KL) loss  T: temperature
     for softening probability distributions"""
      # Soft loss — KL divergence between softened distributions
      soft_student = F.log_softmax(student_logits / T, dim=1)
      soft_teacher = F.softmax(teacher_logits / T, dim=1)
      kl_loss = F.kl_div(soft_student, soft_teacher, reduction='batchmean')*(T ** 2)
      # Hard loss — cross entropy with ground truth labels
      hard_loss = F.cross_entropy(student_logits, labels)
      return alpha * kl_loss + (1 - alpha) * hard_loss
```

B.6 Inference Latency Measurement

```
# Benchmarking utilities — CPU latency and RAM
import time, tracemalloc
def measure_latency(model, n_runs=100, n_warmup=20, device='cpu'):
	model.eval().to(device)
	dummy = torch.randn(1, 3, 32, 32).to(device)
	with torch.no_grad():
		# Warmup — discard timing from cold start
		for _ in range(n_warmup):
			_ = model(dummy)
		# Timed runs
		latencies = []
		for _ in range(n_runs):
			start = time.perf_counter()
			_ = model(dummy)
			latencies.append((time.perf_counter() - start) * 1000) # ms

			return {
			'mean_ms': sum(latencies)/len(latencies),
			'min_ms':  min(latencies),
			'max_ms':  max(latencies)
			}
def measure_peak_ram(model, device='cpu'):
	model.eval().to(device)
	dummy = torch.randn(1, 3, 32, 32).to(device)
	tracemalloc.start()
	with torch.no_grad():
		_ = model(dummy)
		_, peak = tracemalloc.get_traced_memory()
		tracemalloc.stop()
		return peak / (1024 ** 2)  # Convert bytes to MB
```

— END OF REPORT —